

GAM200 - ENGINE

Level Editor Guide

1. Introduction	4
2. Overview of Editor Interface	4
Scene View / Viewport.....	4
Inspector / Property Editor	5
Assets Browser	6
Toolbar (Play, Stop, Save, Load).....	7
Debug Menu	8
Profiler	9
3. Basic Operations	10
3.1 Saving & Loading Levels	10
3.1.1 Saving Levels	10
3.1.2 Loading Levels	10
3.1.3 Creating Levels.....	11
3.1.4 Deleting Levels	11
3.2 Adding and Removing Game Objects	13
3.2.1 Empty Game Objects.....	13
3.2.2 Prefab Game Objects.....	14
3.3 Editing Properties	15
4. Advanced Features	17
4.1 Prefabs	17
4.1.1 Creating prefabs.....	17
4.1.2 Editing prefabs	19
4.2 Play/Stop Simulation	21
4.3 Editor Camera Controls	21
4.4 Object Picking	23
4.5 Undo Redo	24
4.6 Layering System	24
4.7 Tile maps	27
4.7.1 Tilesheet asset creation	27
4.7.2 Creating a Tile map	28
4.8 Scripting	29
4.8.1 Creating scripts	29

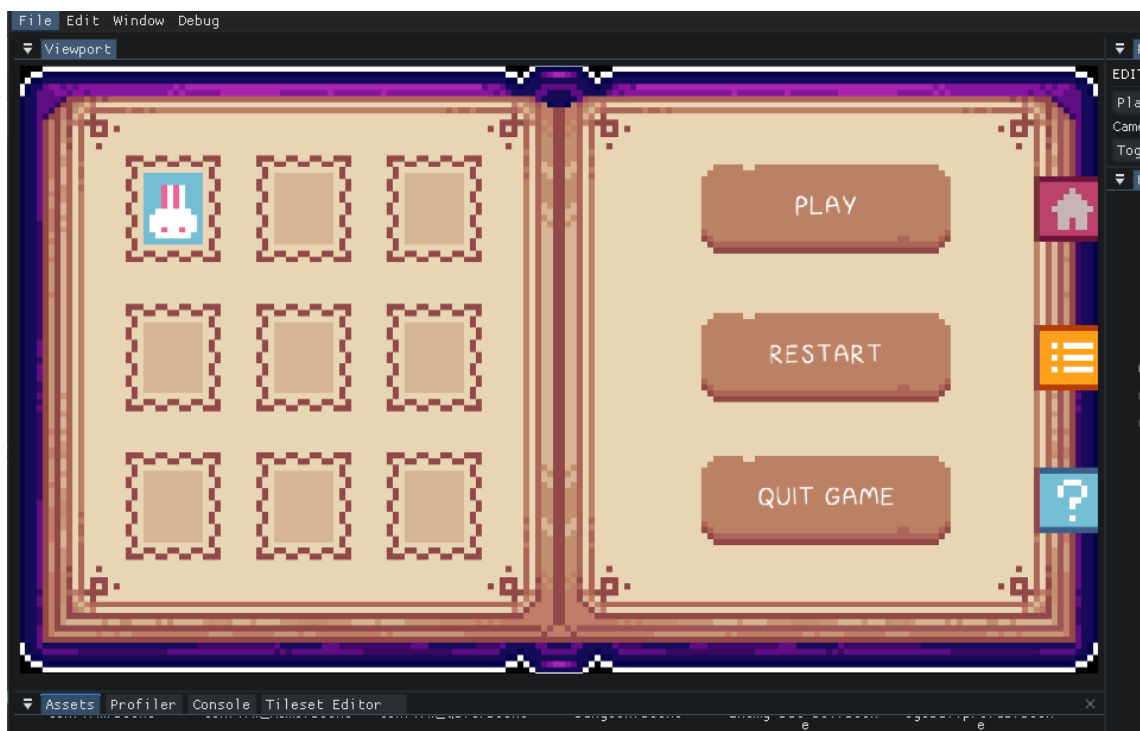
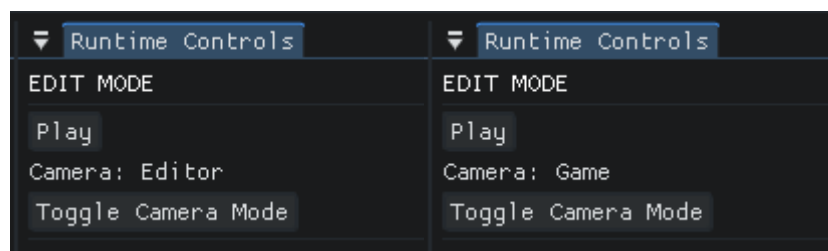
4.9 Lighting.....	33
4.9.1 Adding lighting.....	33
4.9.2 Types of lighting.....	34
4.9.3 Light color.....	34
4.9.4 Light intensity.....	35
4.10 Particle system.....	36
4.11 Video player	39
4.12 Build Size Analyzer.....	40
4.13 Item window	41
5. Asset Management.....	43
5.1 Importing.....	43
5.2 Browsing.....	44
5.3 Replacing Assets	44
6. Troubleshooting / FAQ.....	45

1. Introduction

The Level Editor is a built-in tool in our custom engine that allows designers to create, modify, and test levels without writing code. It supports object placement, property editing, prefab instantiation, and live simulation via Play Mode.

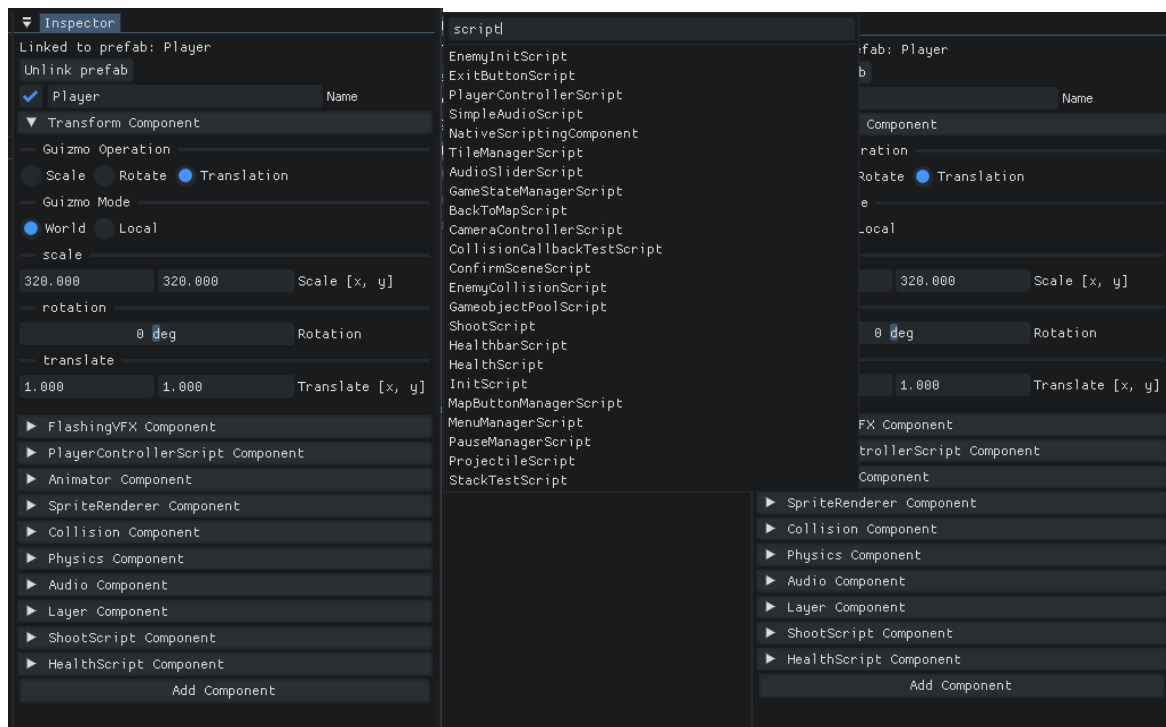
2. Overview of Editor Interface

Scene View / Viewport



Inspector / Property Editor

The **Inspector** (or **Property Editor**) allows users to view and modify all components attached to a selected entity within the scene. Each component listed—such as TransformComponent, SpriteRendererComponent, CollisionComponent, representing a distinct aspect of the entity’s behavior or appearance.

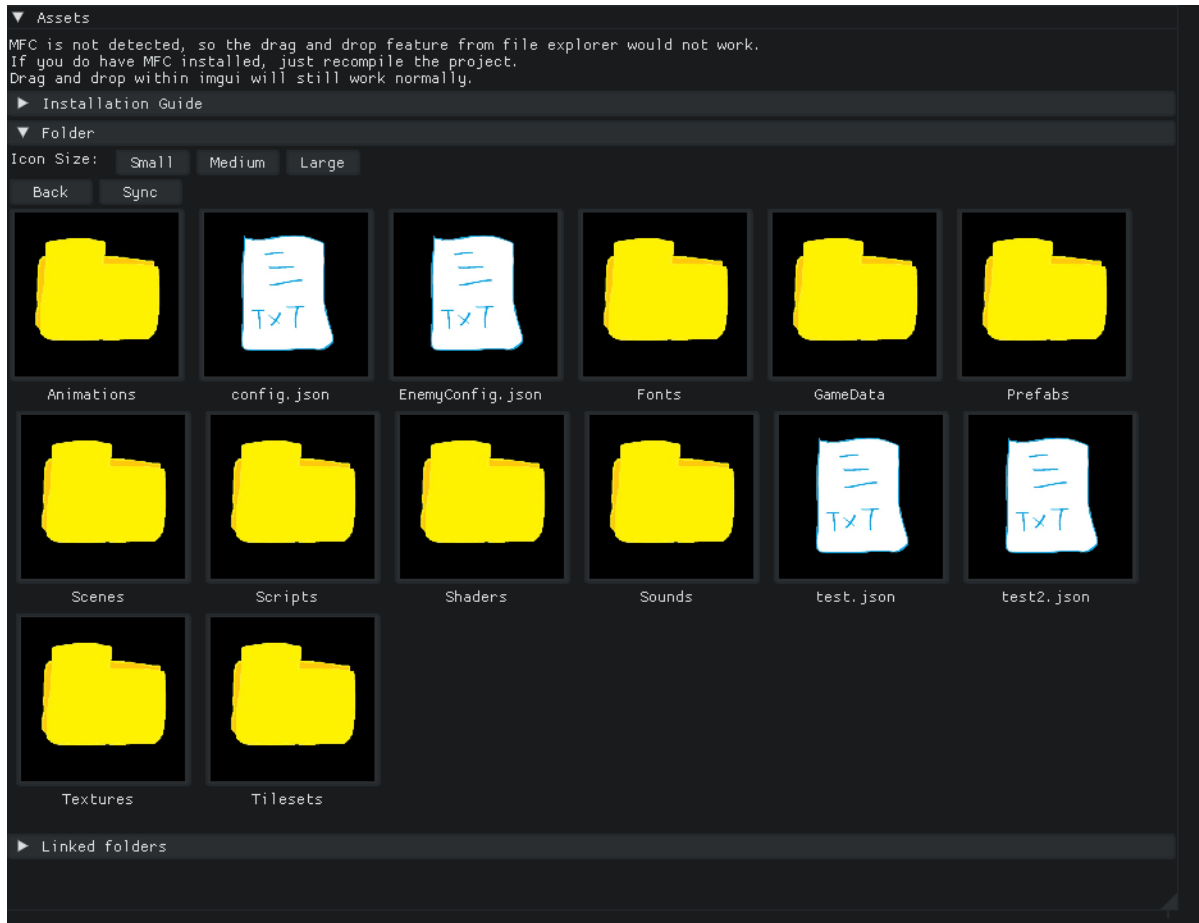


Through this panel, users can add and adjust properties in real time, such as position, rotation, and scale under the TransformComponent, or inspect custom scripts like PlayerController and RotateController to configure their behavior.

This interface provides an intuitive way to interact with the Entity-Component System, enabling developers to test, debug, and fine-tune gameplay elements directly from the editor without recompiling the code.

Assets Browser

The **Asset Browser** provides a visual interface for browsing and managing project assets directly within the editor. It displays all files and folders in the project directory, including textures, sounds, shaders, animations, and configuration files. Users can navigate through directories, change icon sizes for easier viewing, and refresh the asset list to reflect external changes.



While drag-and-drop from the system file explorer requires MFC support, **in-editor drag-and-drop** using ImGui is supported — when a **.png** file is dragged onto an entity that contains a **SpriteRendererComponent**, the entity's texture is automatically updated to the selected image. This feature streamlines the asset assignment process, allowing quick visual updates without manually editing component properties.

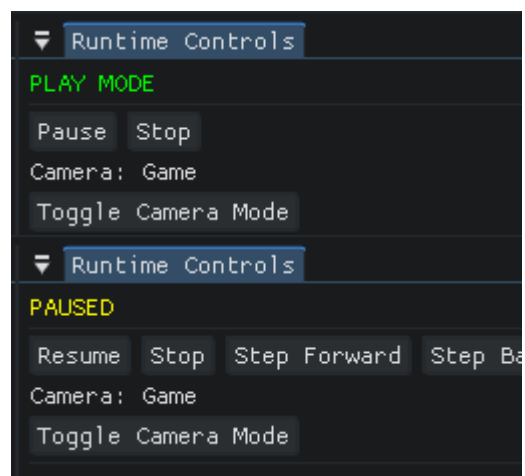
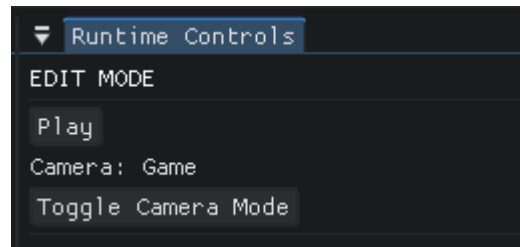
Toolbar (Play, Stop, Save, Load)

The **Toolbar** provides essential runtime controls that allow users to manage the simulation state of the engine. It includes the **Play**, **Pause**, and **Stop** functions, as well as additional step controls for debugging.

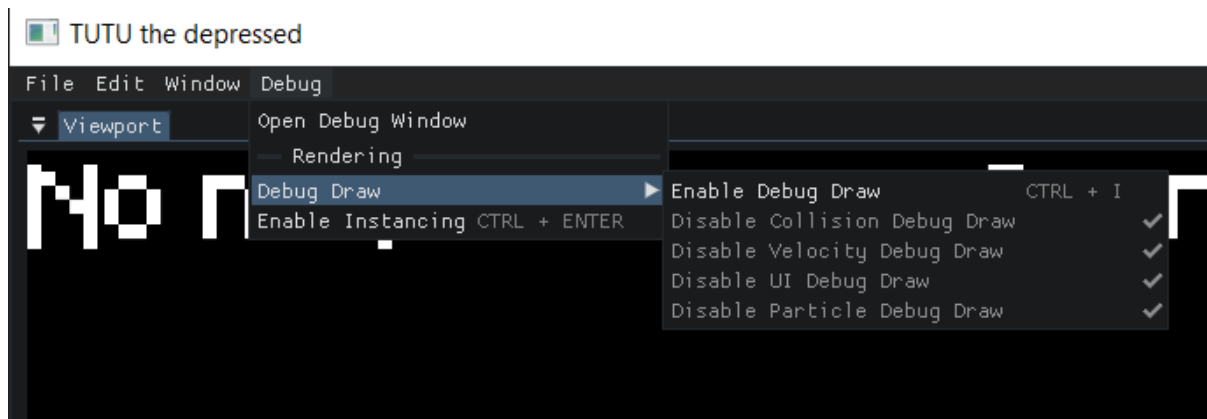
In **Edit Mode**, users can modify the scene freely and press **Play** to enter **Play Mode**, where all game systems such as physics and scripting begin running in real time.

While in **Play Mode**, the simulation can be paused at any moment using the **Pause** button or completely stopped to return to **Edit Mode**.

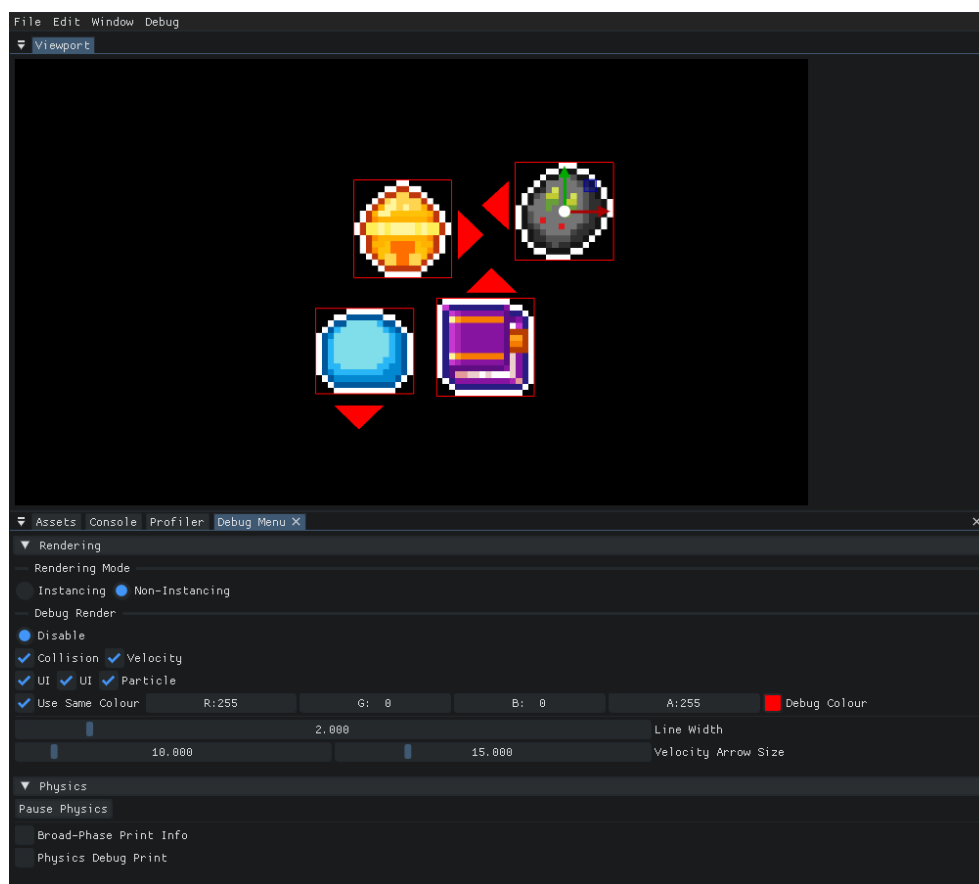
When the simulation is paused, the user can either resume the simulation from where they paused, or stop the simulation and reset the scene to its original state. The **Step Forward** and **Step Backward** buttons are displayed but not currently working.



Debug Menu



The Debug Tab on the top of the application allows quick access to certain debug features such as: (1) toggling different debug draws; (2) toggling render mode (between instance and non-instanced). For more debug options, open the Debug Window.



The Debug Menu provides more customisability for debug rendering, such as different line width, size, and colour.

Under the Physics tab, users would be able to pause and step through the physics system as well as toggle specific physics print information.

Profiler

The **profiler** shows runtime information of different systems in the engine allows users to **diagnose** the portions of their code that is underperforming, a **graph** is provided to show how **performance** fluctuates over time and how different systems might affect each other. More **details** are also provided under values such as the average processing time and max processing time.



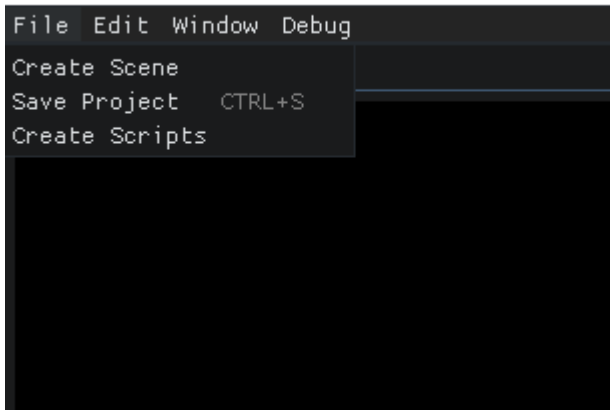
3. Basic Operations

3.1 Saving & Loading Levels

3.1.1 Saving Levels

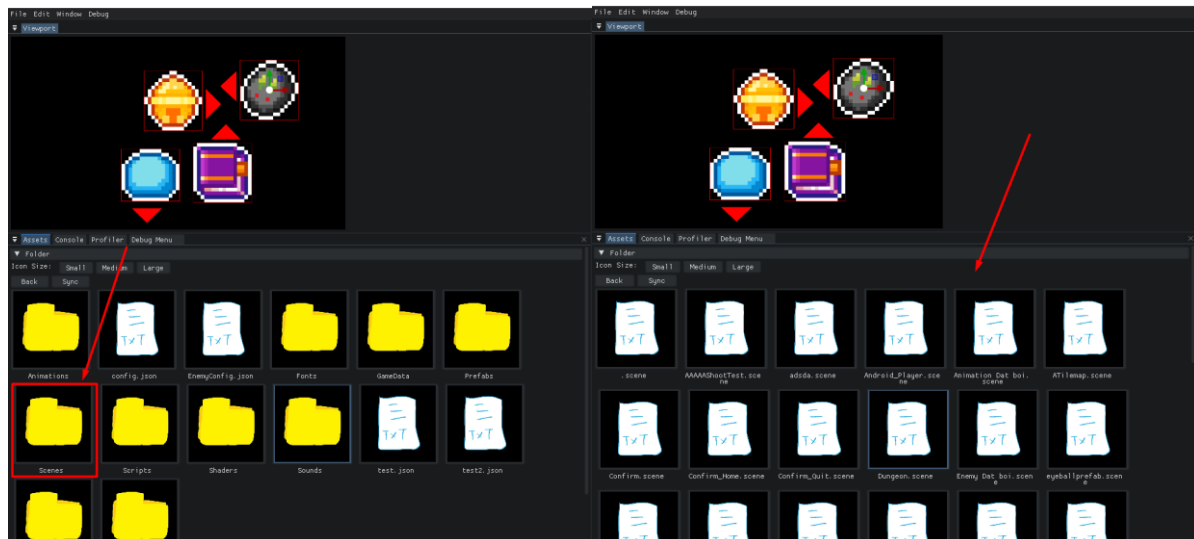
Saving of levels is done by clicking on the “Save Project” button in the Scene Select window or pressing Ctrl+S.

 TUTU the depressed



3.1.2 Loading Levels

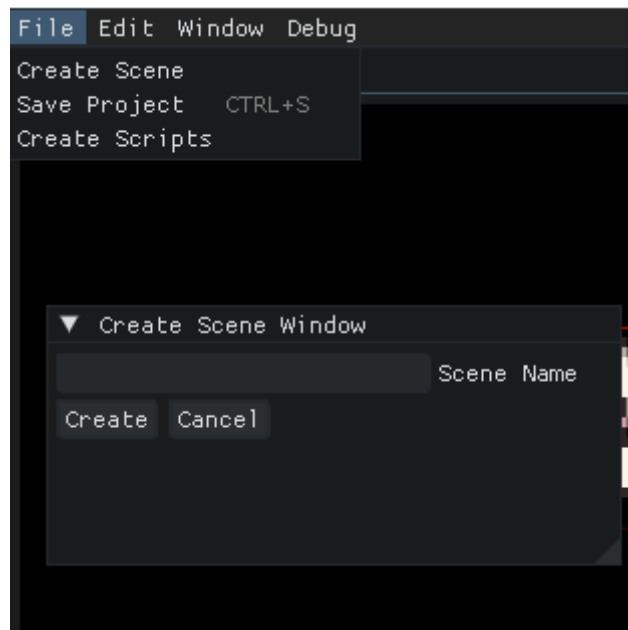
Loading of levels is done through asset browser. The user can navigate into the scenes folder and all view the name of all the scenes. Clicking on any of these scenes will unload the currently loaded scene and load the clicked one. Clicking the scene that is currently loaded will reload the scene.



3.1.3 Creating Levels

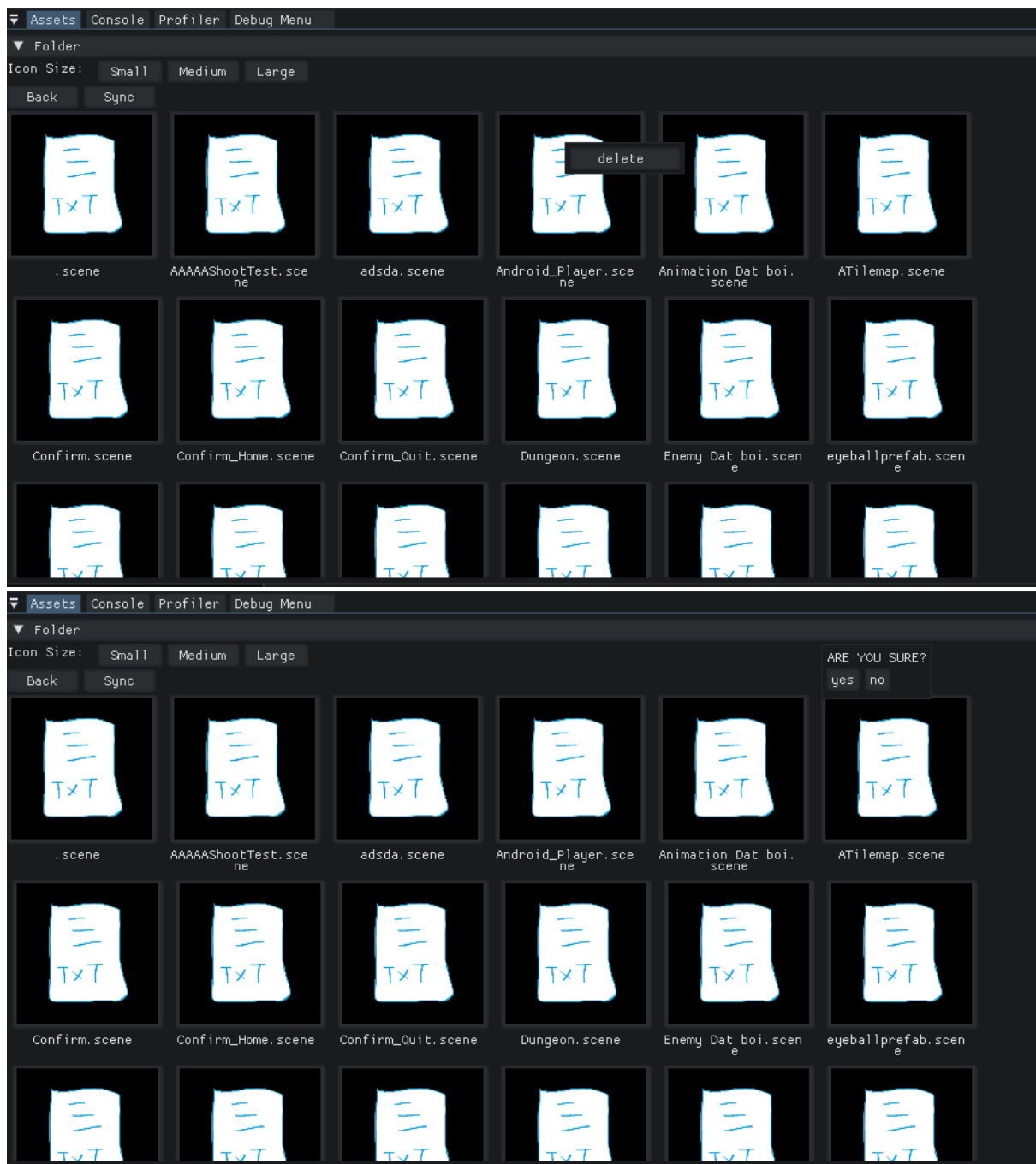
Under the file tab in the menu bar at the top, the user can click on the create scene menu item.

A pop-up window will appear for the user to create a new scene. Any errors when creating a new scene will reflect in the console log



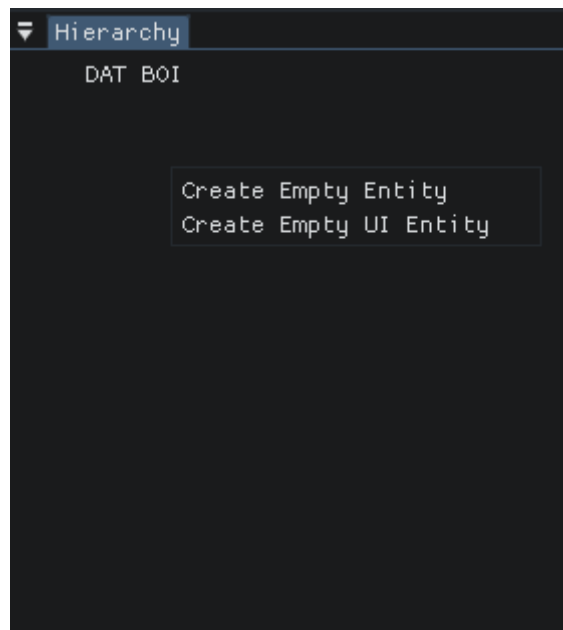
3.1.4 Deleting Levels

As with all other assets, the user can right click on the scene in the asset browser and delete it from there.



3.2 Adding and Removing Game Objects

3.2.1 Empty Game Objects



1. Right click anywhere on the hierarchy, window and a popup should appear
2. To create an entity the user simply has to:
 - Click on “Create Empty Entity” to create a normal entity
 - Click on “Create Empty UI Entity” to create an entity for UI elements

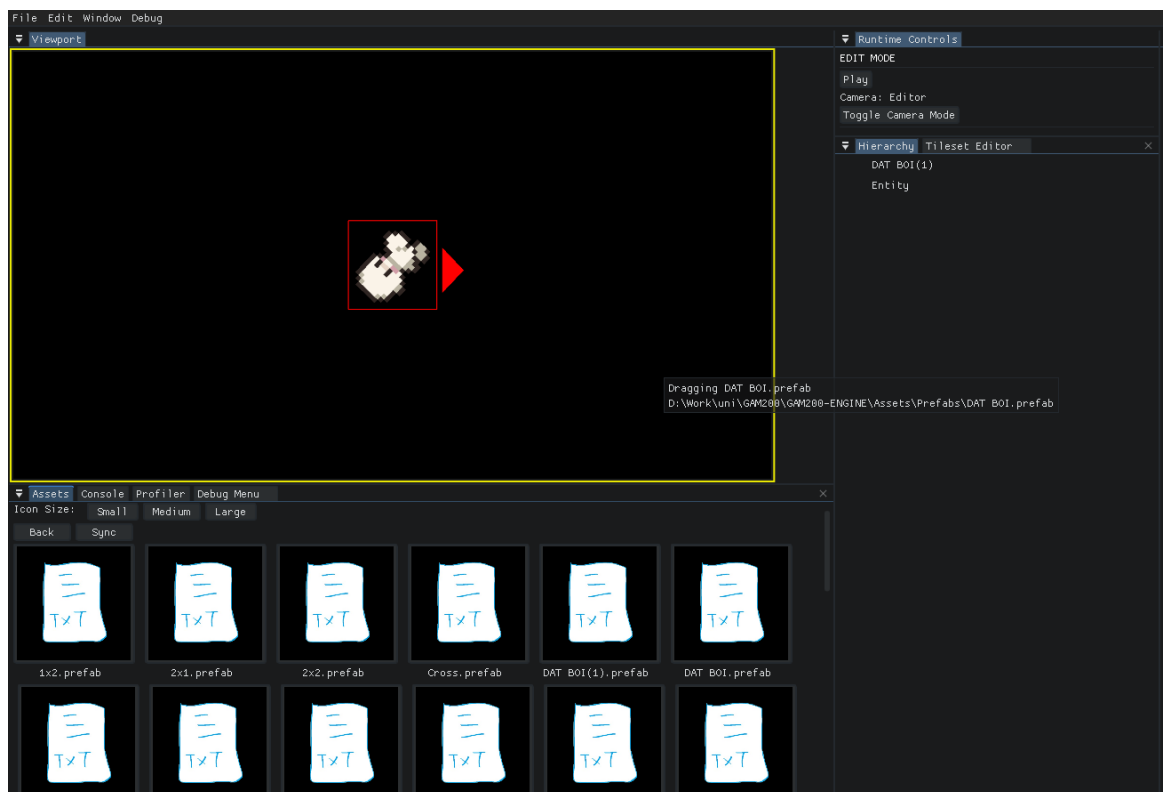
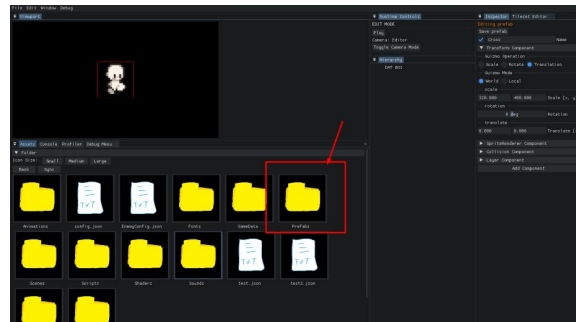
To delete an entity,

1. Select an entity by clicking on the desired entity on the Viewport Window or Hierarchy window
2. Press the [Del] hotkey.

3.2.2 Prefab Game Objects

Step-by-step example for a prefab to be inserted:

1. Open the **Prefab** folder in the assets browser.
2. Drag any prefab from the assets folder into the viewport
3. An instance of the prefab should be created and reflected in the hierarchy window

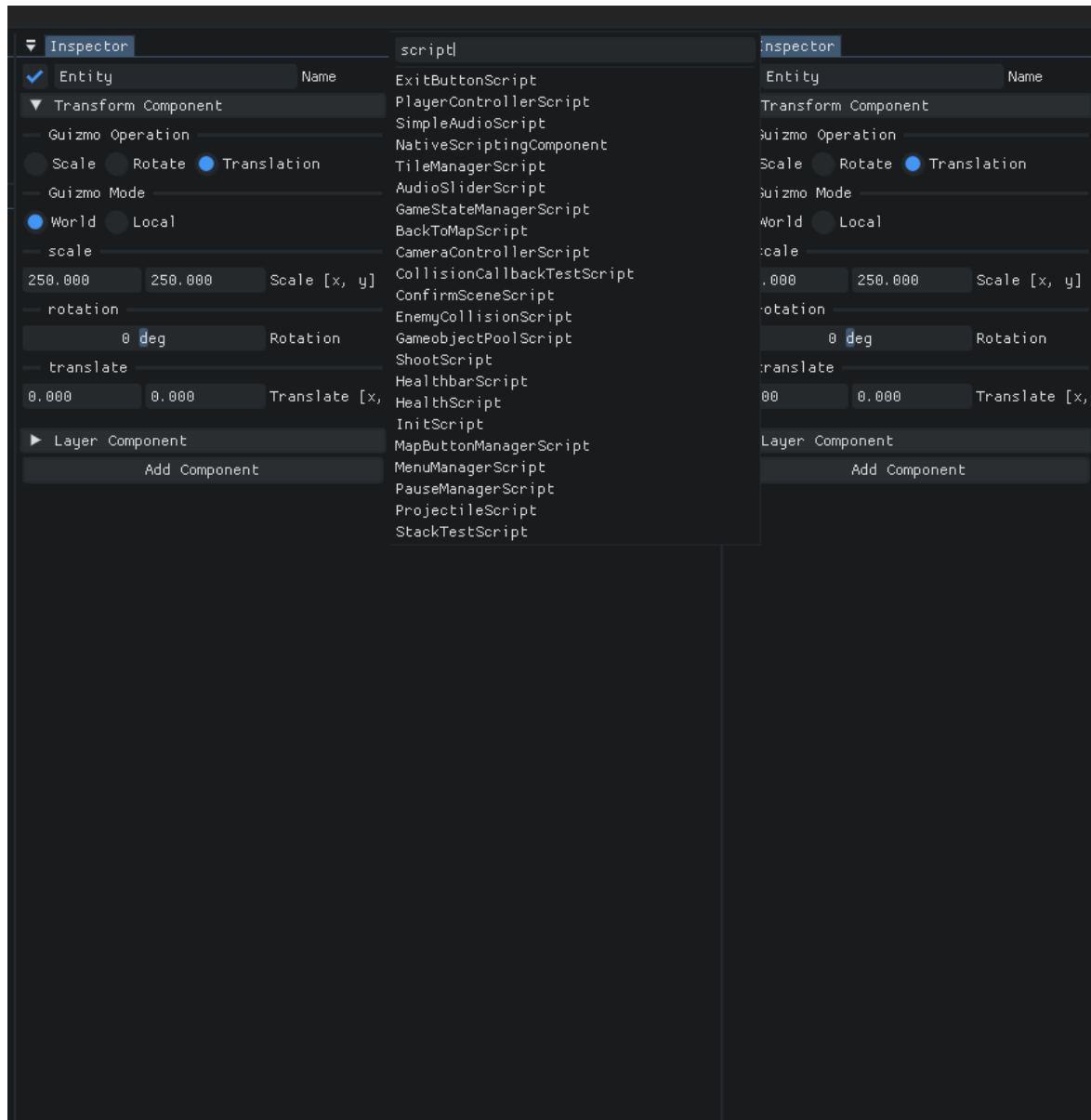


Instantiated prefabs can be identified by a lighter (prefab) tag to the right of the entity names

3.3 Editing Properties

Objects can be selected either via the hierarchy panel or by clicking on the desired object in the Viewport window. After an object has been selected, their properties can be viewed and modified via the **Inspector Panel**.

Below is what the inspector window looks like when selecting a new empty object



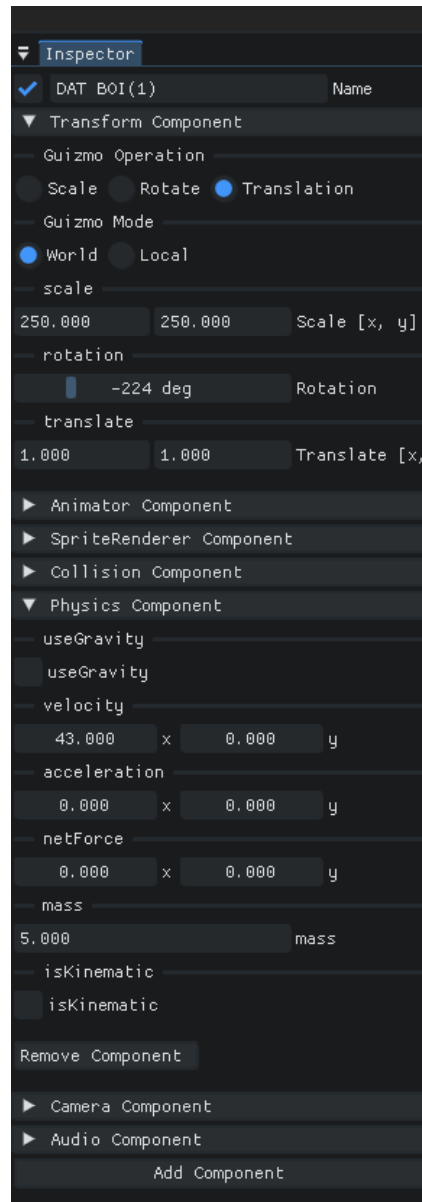
And below is what it looks like with components:

Step-by-step example for a prefab to be inserted:

Each component can be manually edited by interacting with the widgets

Editable properties include:

- Transform (Position, Rotation, Scale)
- Sprite/Texture
- Physics/Collider
- Scripts or AI components
- And any other components that are reflected when loaded into the componentRegistry



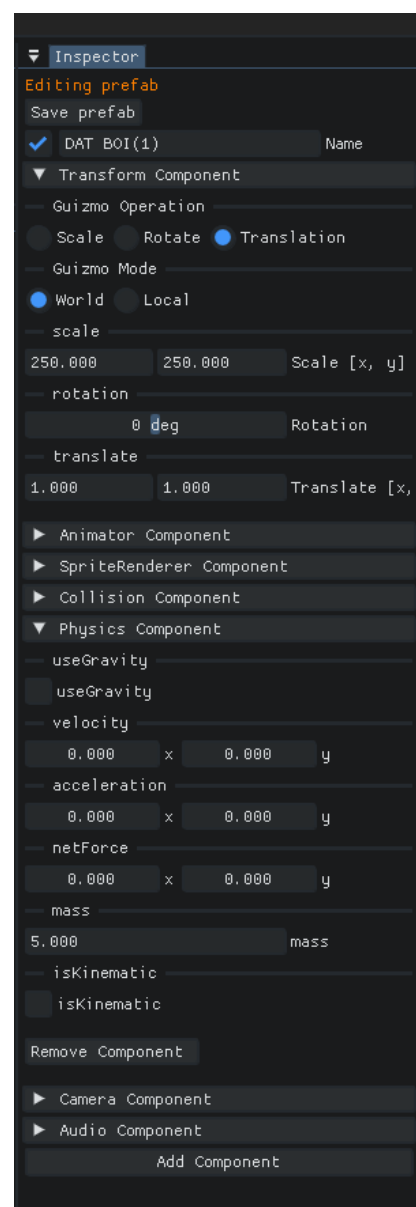
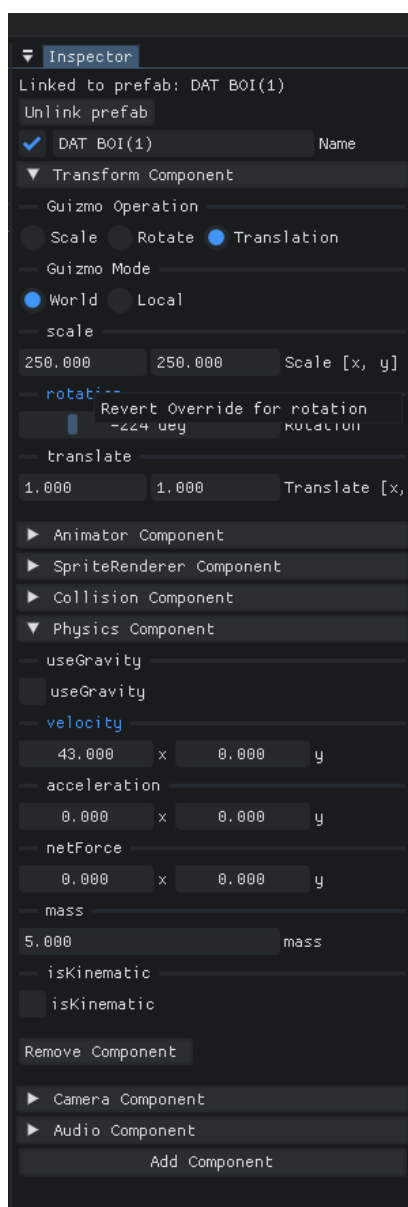
4. Advanced Features

4.1 Prefabs

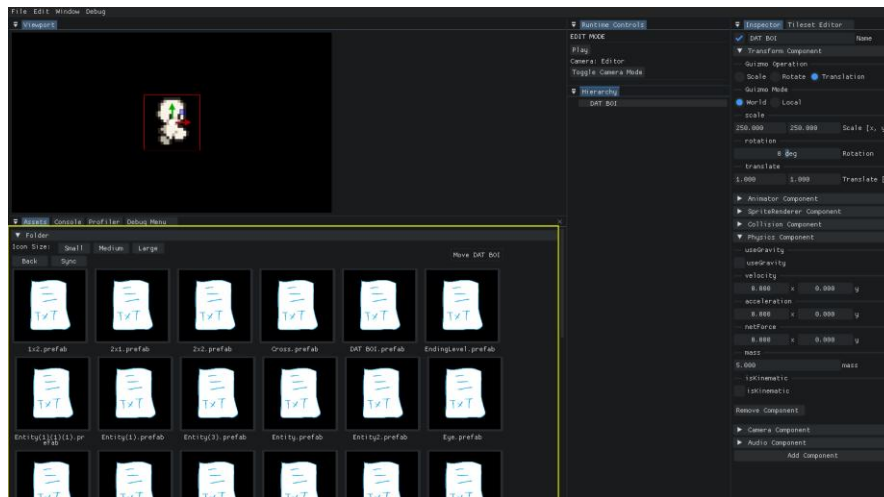
4.1.1 Creating prefabs

Prefabs are custom user-defined game objects with its components and children pre-defined. Users are able to instantiate them in the scene through the editor as well as during game runtime using scripts

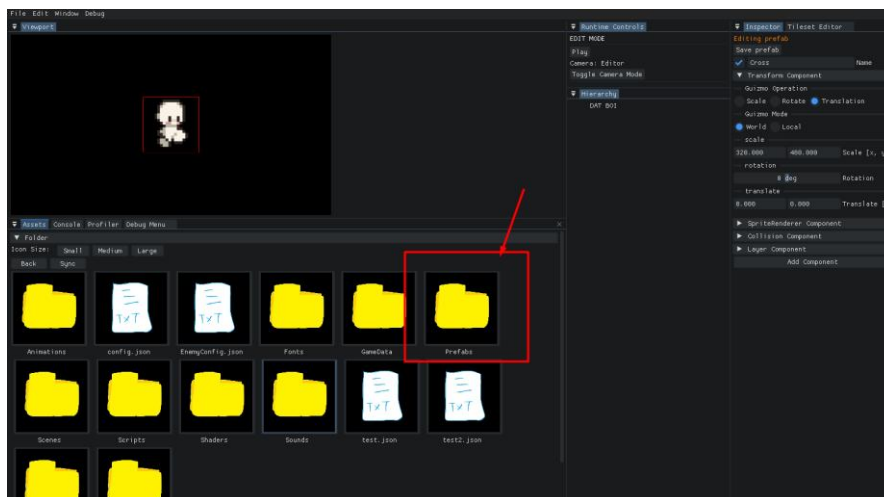
The 2 images below are what the inspector will display when inspecting a prefab.



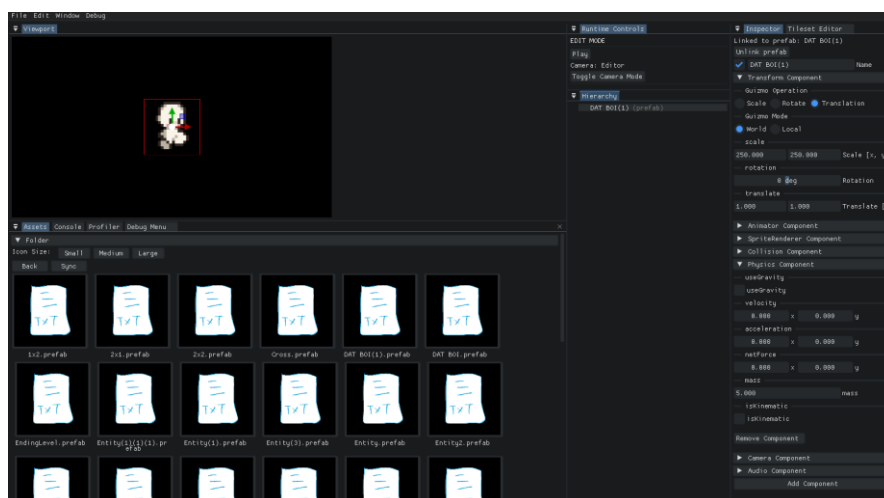
To create a prefab, the user can drag an existing entity from the hierarchy window at any time straight into the asset browser.



Dragging an entity into the asset browser will create new file in the prefabs folder

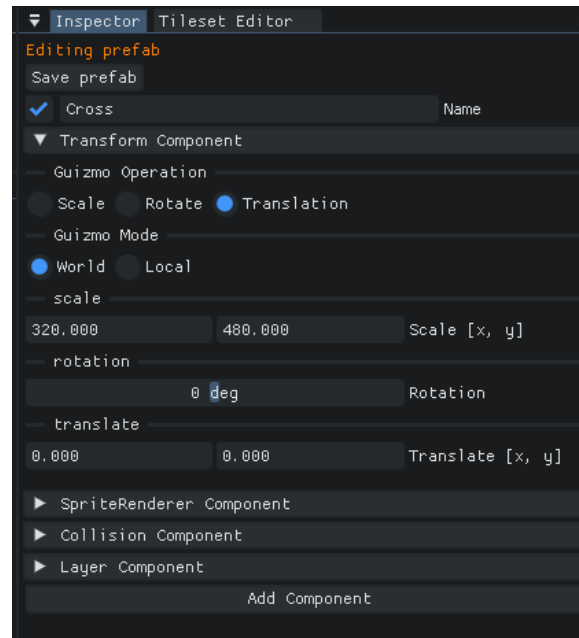


The object the entity is based on will also be linked up to the prefab, allowing any changes done to the prefab to be reflected upon reloading a scene.



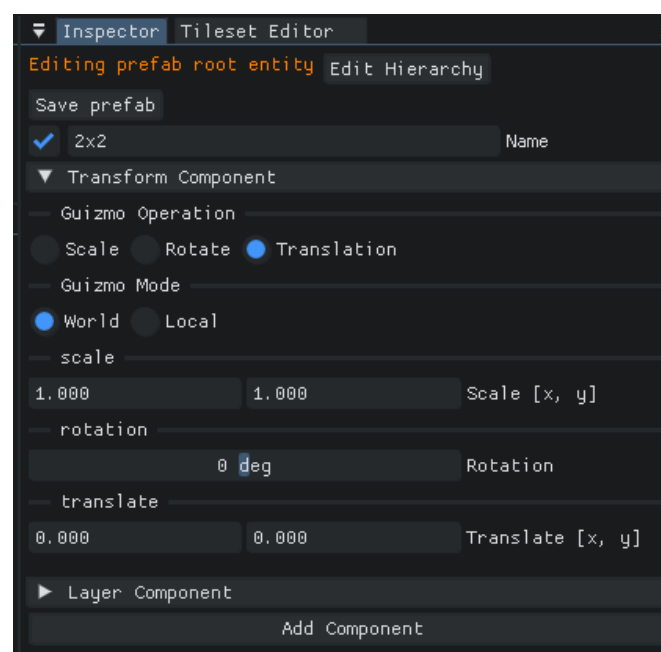
4.1.2 Editing prefabs

To edit a prefab, the user can click on any prefab file in the asset browser. The prefab's data will then be displayed in the inspector window with the orange "Editing prefab" text above.

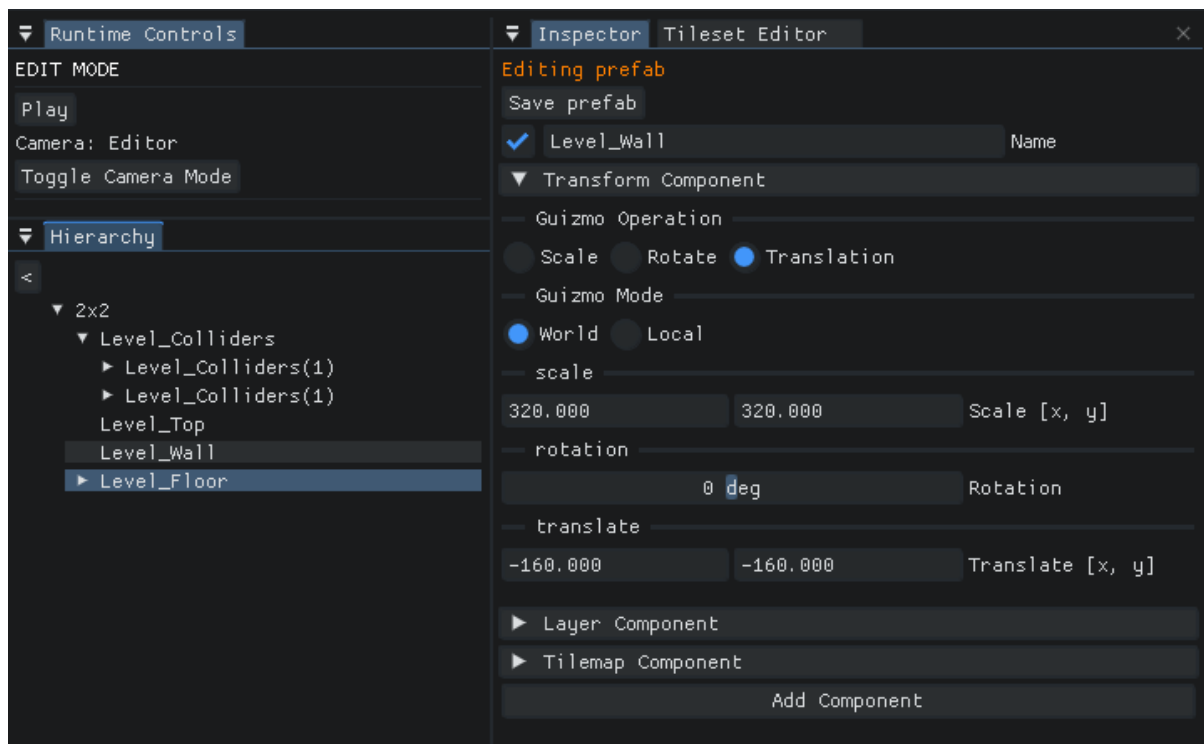


To save any changes to the prefab, the user can click the "Save prefab" button at the top, saving the prefab. Saving the prefab will allow its changes to propagate throughout all scenes. However, for propagation to work in the current scene, the user will have to reload the currently loaded scene.

When editing prefabs with children, there will be an additional "Edit hierarchy" button at the top of the inspector.



Clicking the button will cause the hierarchy window to switch to displaying only the hierarchy of the prefab, where the user can then choose which entity to edit. Clicking on the save button will save the entire entity hierarchy.

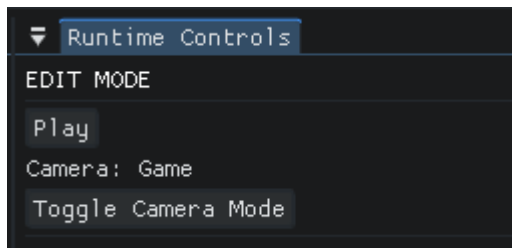


To exit prefab editing mode, the user just needs to click on the “<” back button at the top left of the hierarchy window to revert to displaying scene data.

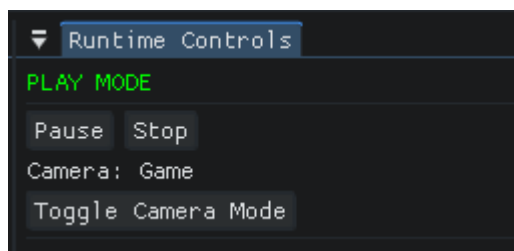
4.2 Play/Stop Simulation

Accessed from the Runtime Controls window:

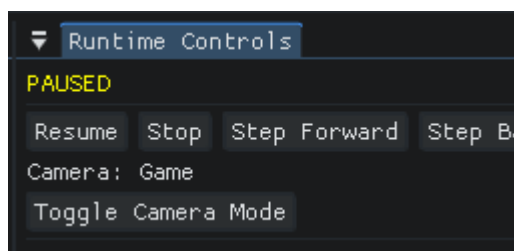
- **Play** button: Starts game simulation.



- **Stop** button: Resets to pre-play state using the SaveState system. Include a screenshot of the toolbar.
- **Pause** button: Stops game simulation.



- **Resume** button: Starts the game simulation continuing from the play state before game was paused
- **Step forward** button: while paused, allows user to run the simulation one update iteration at a time
- **Step backward** button: while pause, allows user to run the simulation one update iteration at a time, however, the time delta supplied is negative, effectively causing all systems to update backwards



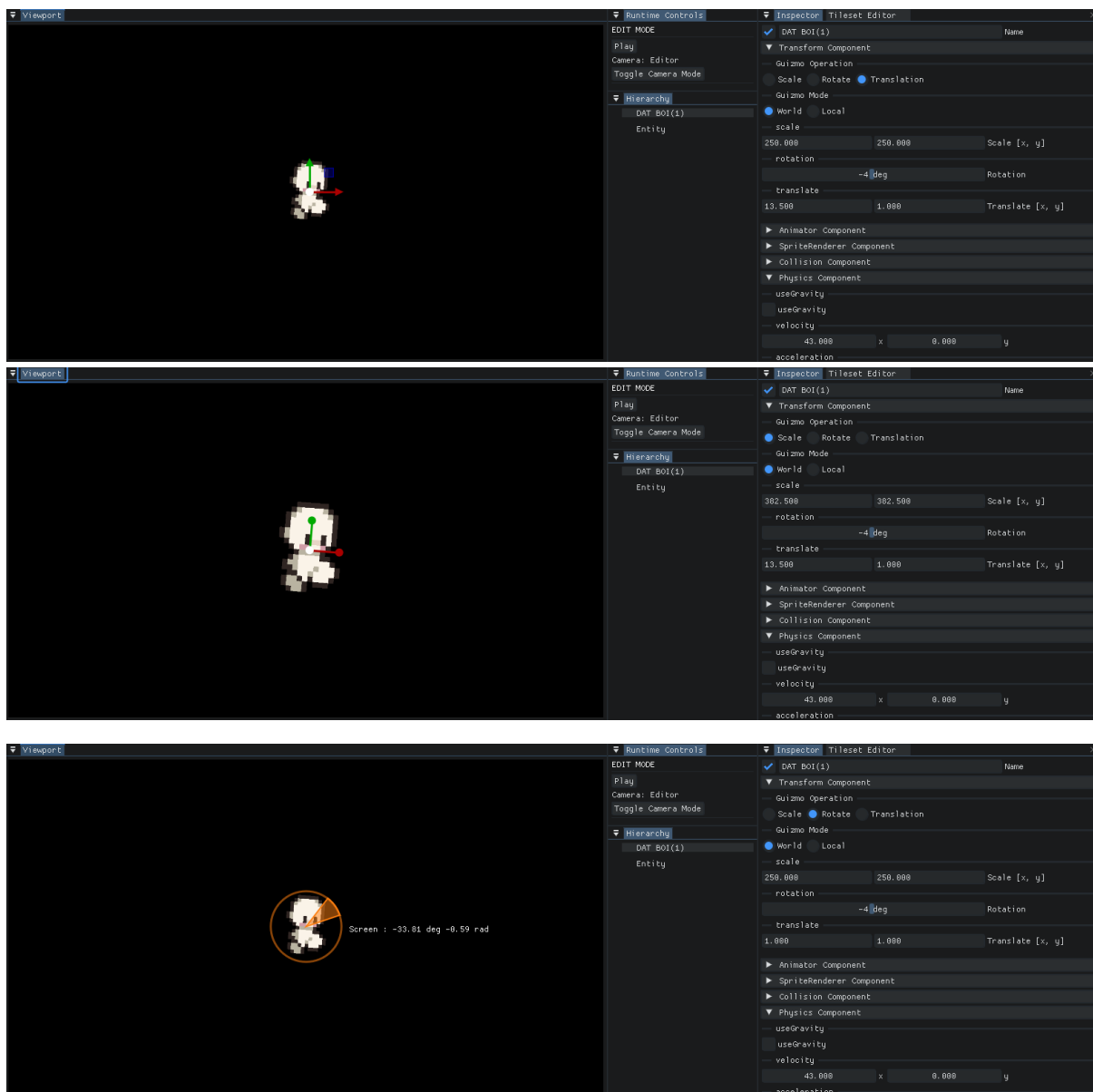
4.3 Editor Camera Controls

- Right-click + drag: pan camera

- +/-: zoom in/out
- Scroll Wheel: zoom in/out
- Y: Toggle between editor camera and game camera

4.4 Object Picking

Objects can be selected, for viewing and real time editing of their components, simply by left clicking on the desired object on the Viewport window with the Inspector panel open / active. If the selected object contains a TransformComponent, it can be manipulated with ImGuizmos controls – for scaling, rotating, or translating position. To switch between the 3 ImGuizmos modes, it can either be done through hotkeys ([CTRL + E] for scale, [CTRL + R] for rotation, [CTRL + T] for translation) or under the TransformComponent in the Inspector Panel. Otherwise, editing the object can be achieved through the Inspector panel.



4.5 Undo Redo

Certain actions in the editor can be undone/redone by pressing [CTRL + Z] and [CTRL + Y] respectively, these actions are currently limited to most component values and adding/removing entities but not available for adding/removing yet as of M3 due to instability issues.

4.6 Layering System

A layering system has been implemented to alter how rendering and collision interactions.

In [LayerComponent], there are 3 fields available for editing:

- [renderPriority]: value that affects what gets drawn on top (lower value is drawn on top of higher value).
- [layer]: the layer that the entity is currently residing on.
- [layers interactions]: the layers that the entity's layer can interact with (for collision).

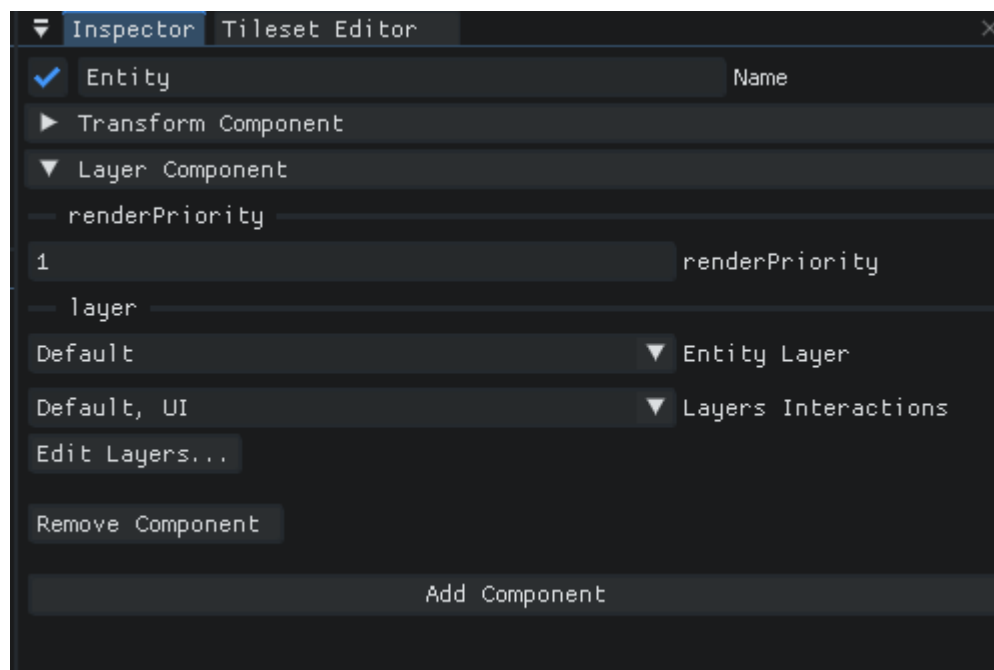
In [CameraComponent], there exist a field that interacts with the layering system:

- [cullingMask]: Choose the layers to render when using Game camera (only works if the current CameraComponent has the flag [isMainCam] flag set to true).

There is also an Edit Layer window, which can be accessed via the [Edit Layers] button within the LayerComponent, that provides more granular configurations for the layers such as:

- changing layer names
- enabling layers, done by giving an empty layer a name. Layers without a name are treated as disabled layers.
- Changing the layer interactions

After making the amendment, the [Save] button will serialize and save the layer information to the config.json.

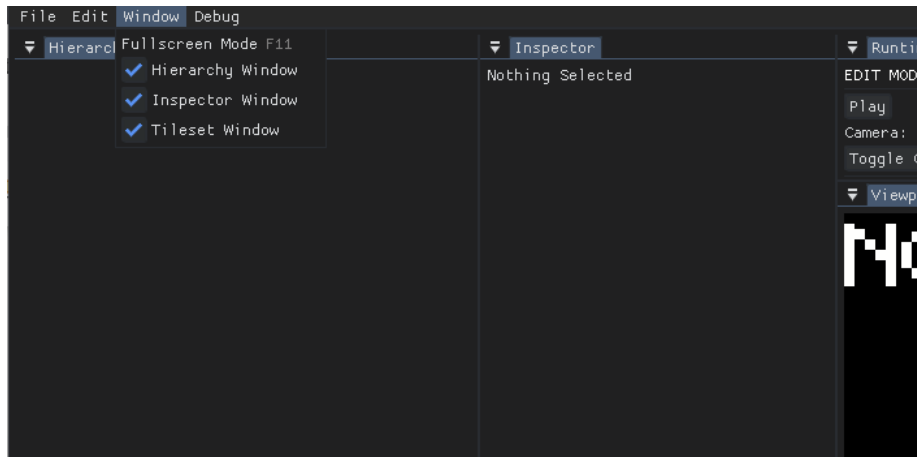


Layers Configuration				
Layer No.	Layer Name	Layer Interactions		
Layer 0	Default	Default, UI		
Layer 1	UI	Default, UI		
Layer 2	Player	Player, Enemy, EnemyProject		
Layer 3	Enemy	Player, Enemy, PlayerProjec		
Layer 4	PlayerProjectile	Enemy, Environment		
Layer 5	EnemyProjectile	Player, Environment		
Layer 6	Environment	Player, Enemy, PlayerProjec		
Layer 7	Player1	Player1, Enemy0		
Layer 8	Enemy0	Environment, Player1, Enemy		
Layer 9	Enemy1	Enemy1		
Layer 10	Enemy2	Enemy2		
Layer 11	Enemy3	Enemy3		
Layer 12	Enemy4	Enemy4		
Layer 13	Enemy5	Enemy5		
Layer 14	Enemy6	Enemy6		
Layer 15	Enemy7	Enemy7		
Layer 16	Enemy8	Enemy8		
Layer 17	Enemy9	Enemy9		
Layer 18	Bruh	Bruh		
Layer 19	Test	Test		
Layer 20	PauseSettings	PauseSettings		
Layer 21	PauseFAQW	PauseFAQW		
Layer 22	Confirmation	Confirmation		
Layer 23	PauseBG	PauseBG		
Layer 24	PauseFAQA	PauseFAQA		
Layer 25				
Layer 26				
Layer 27				
Layer 28				
Layer 29				
Layer 30				
Layer 31				
Layer 32				
Layer 33				

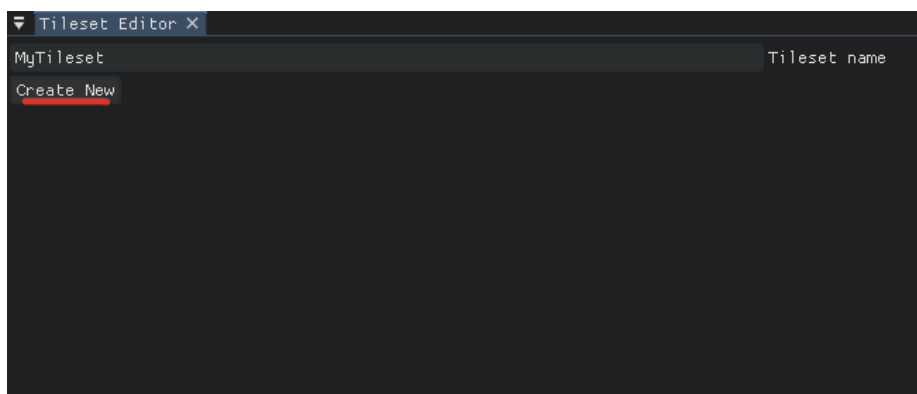
4.7 Tile maps

4.7.1 Tileset asset creation

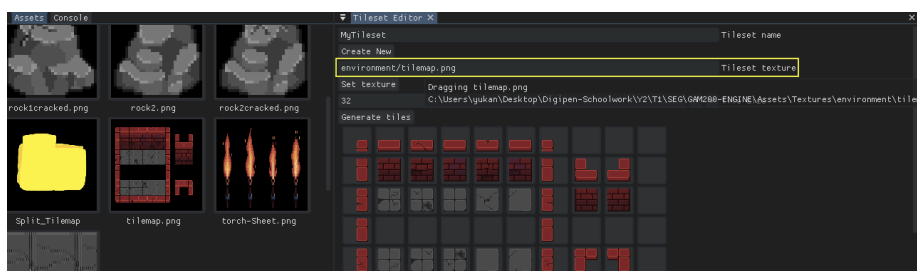
Enable the tileset window from the top tab of the engine



Give your tileset a name and press “Create New” to begin editing



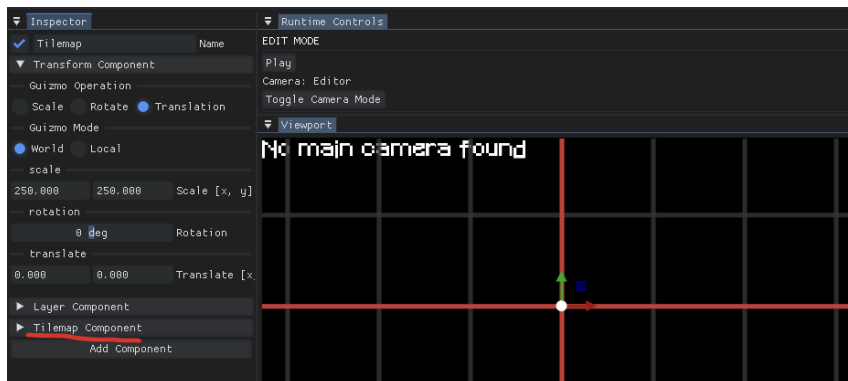
Drag the texture asset you want to make a tileset of into the “Tileset Texture” field



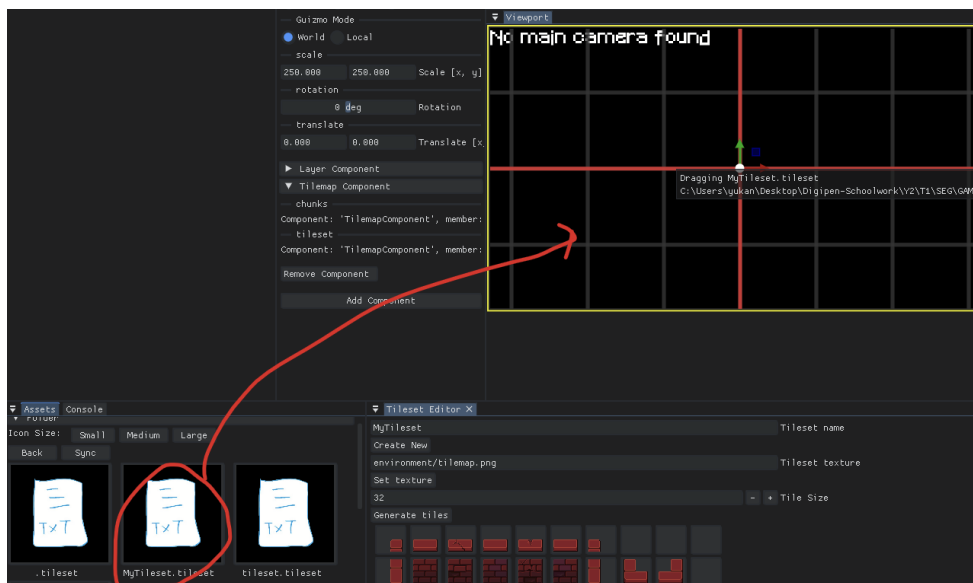
Set the desired tile size and you can begin painting your tilemap

4.7.2 Creating a Tile map

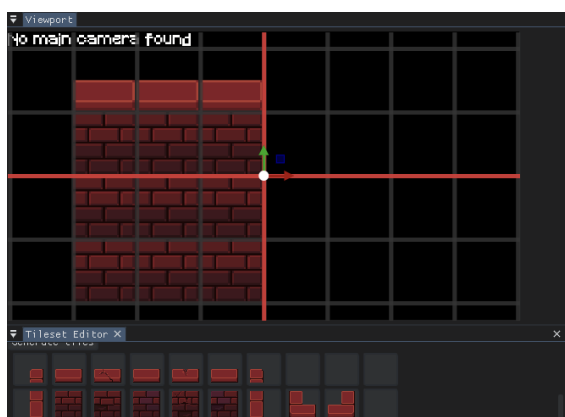
Create a entity with a tilemap component, you should see a grid appear in your editor viewport



With the entity selected, drag your tileset into the editor viewport to assign the tileset to the tilemap component



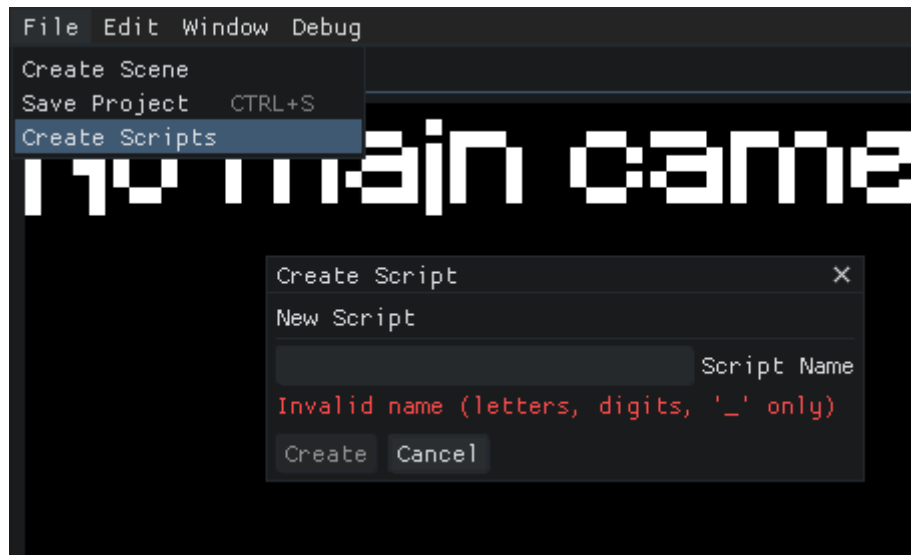
You can now select the tiles from the tileset editor window and begin painting with left mouse click or erase with right mouse click



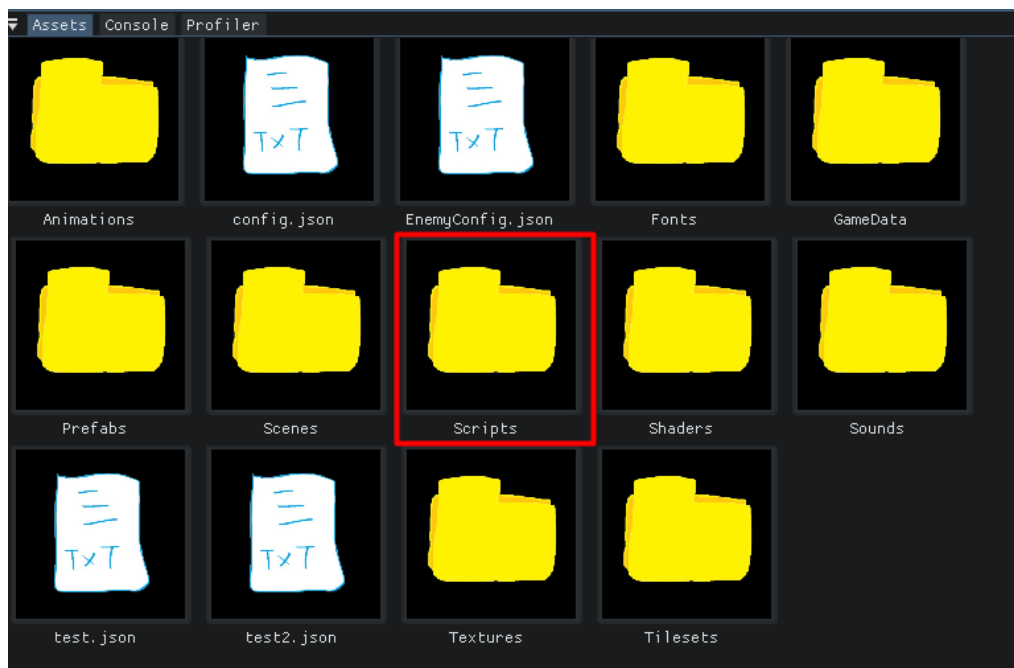
4.8 Scripting

4.8.1 Creating scripts

Users can create scripts via the “file” menu item in the menu bar at the top of the window. Clicking on “Create Scripts” will open a window where users can name and create a new script

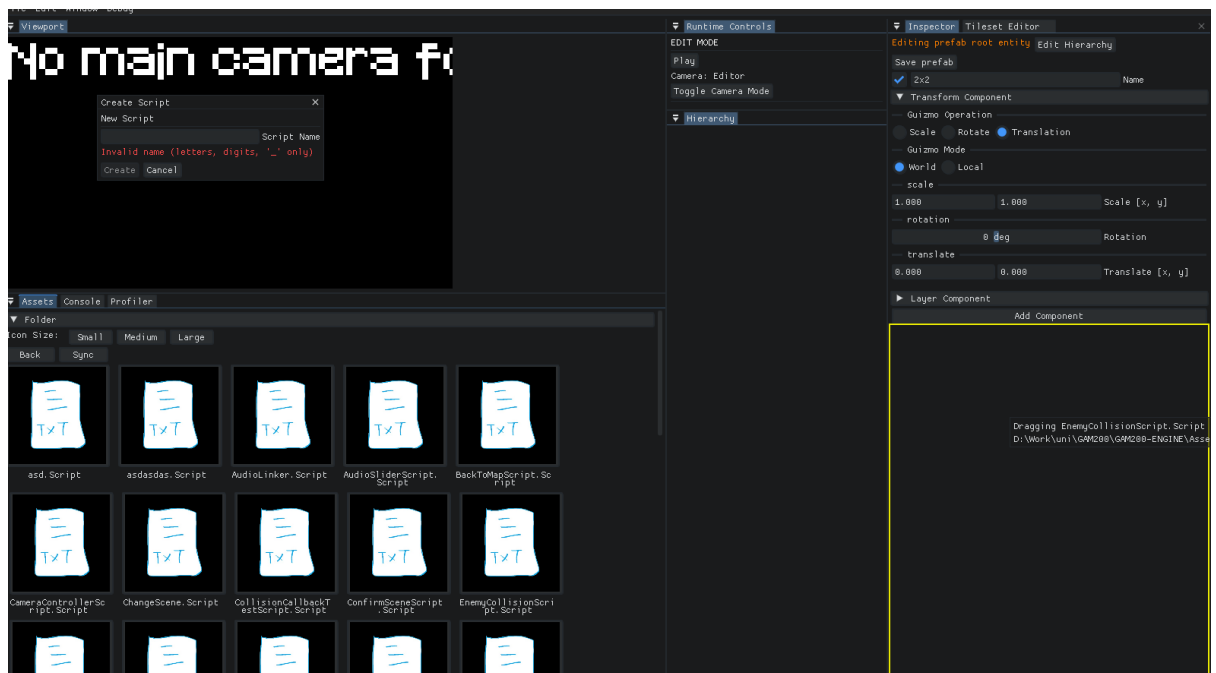


Created scripts will show up in the scripts folder

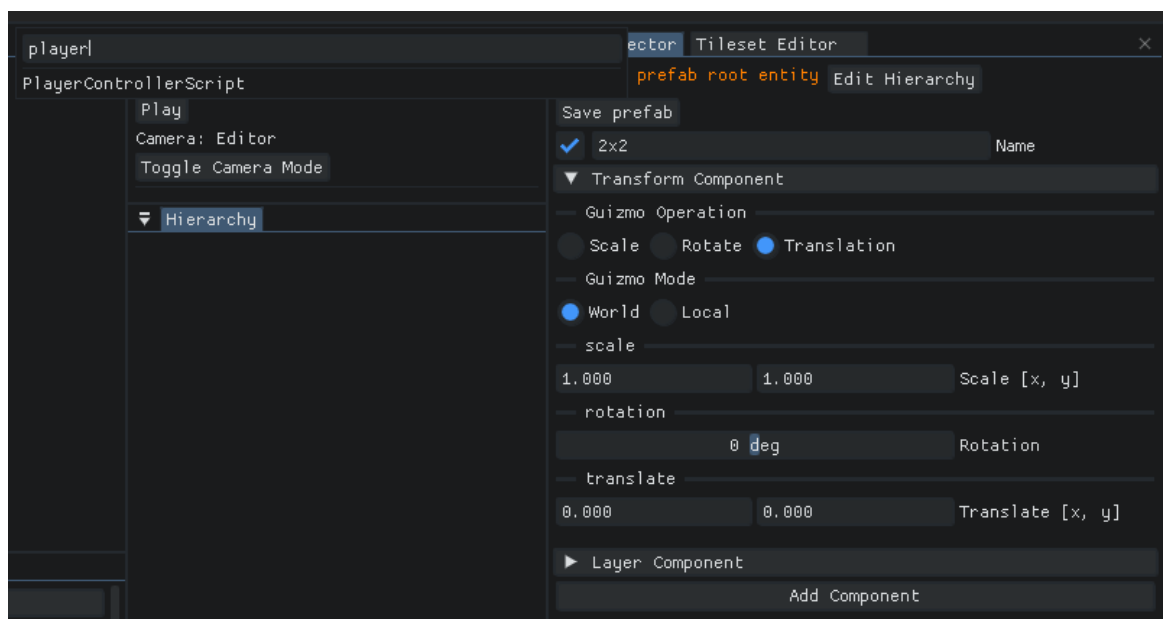


Users can then add scripts via 2 ways

1. Dragging the script from the asset browser into the inspector window



2. Using the “Add component” button in the inspector window and attaching it from there



The created script will show up in the MSVC solution with the given name has both a .h and .cpp file.

```

HealthbarScript.cpp  HealthbarScript.h  X
Scripts  (Global Scope)
7
8
9  class HealthbarScript: public ScriptInstance
10 {
11 public:
12     void BindFrom() {
13         GetComponent<HealthbarScript>(*CEO::Get<Registry>())->entity = entity;
14         *this = *GetComponent<HealthbarScript>(*CEO::Get<Registry>()); };
15     void BindTo() { *GetComponent<HealthbarScript>(*CEO::Get<Registry>()) = *this; };
16
17     std::vector<GameObject> HealthGameObjects;
18     GameObject HealthPrefab{};
19     GameObject TargetGameObject{};
20     float HealthPadding{};
21     TextureObj* EmptyHeartTexture{};
22     TextureObj* FullHeartTexture{};
23     TextureObj* HalfHeartTexture{};
24
25     void OnStart(Registry& registry);
26     void OnUpdate(Registry& registry,float dt, bool firstframe);
27     void OnFixedUpdate(Registry& registry,float dt, bool firstframe);
28
29     void AddHealthGameObject();
30     void UpdateHealthbar(int currHealth, int maxHealth);
31
32     REFLECTABLE_PROPERTIES;
33 };
34 REFL_AUTO(
35     type(HealthbarScript),
36     field(HealthPrefab),
37     field(HealthPadding)
38 )
39

```

By default, a script will have the following functions:

- BindFrom() and BindTo()
 - an internal function to bind the script to the entity
- void OnStart(Registry& registry);
 - User defined function which is run once per startup
- void OnUpdate(Registry& registry,float dt, bool firstframe);
 - User defined function that runs every game loop
 - Runs every delta time increment
- void OnFixedUpdate(Registry& registry,float dt, bool firstframe);
 - User defined function that runs every game loop
 - Runs every frame

Users can also define other functions that can be called from other scripts if required.

A script can also have different properties that can be inspected and set in the editor's inspector window. For data to be displayed in the editor:

1. User declares member variables

2. In the

```
REFL_AUTO(  
type(NameOfScript)  
)
```

Macro, the user needs to register the member variables they want displayed in the editor like this

```
REFL_AUTO(  
type(NameOfScript),  
field(memberName)  
)
```

Do note that the number of members that can be registered is limited to 100 member variables.

There are also a few other event based functions that users can define (All functions are called when another collider entity collides with this entity) :

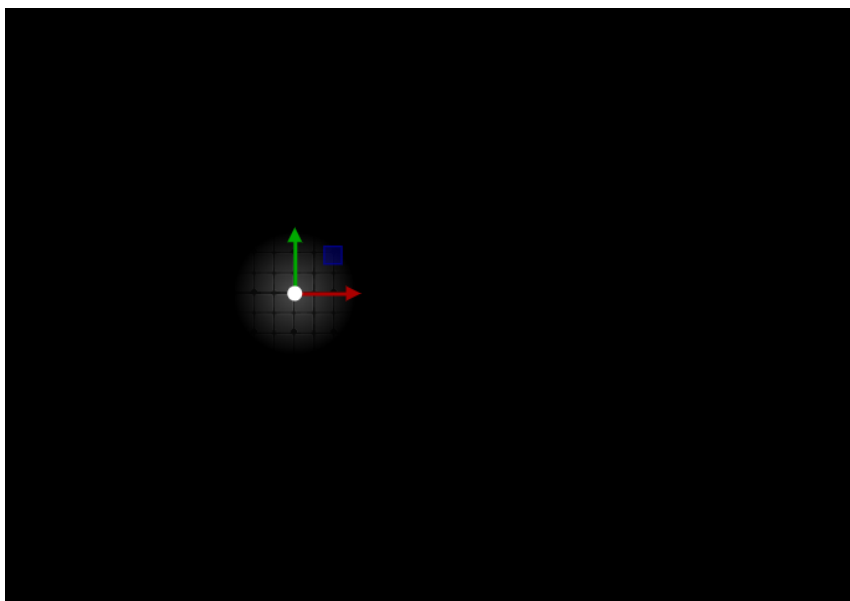
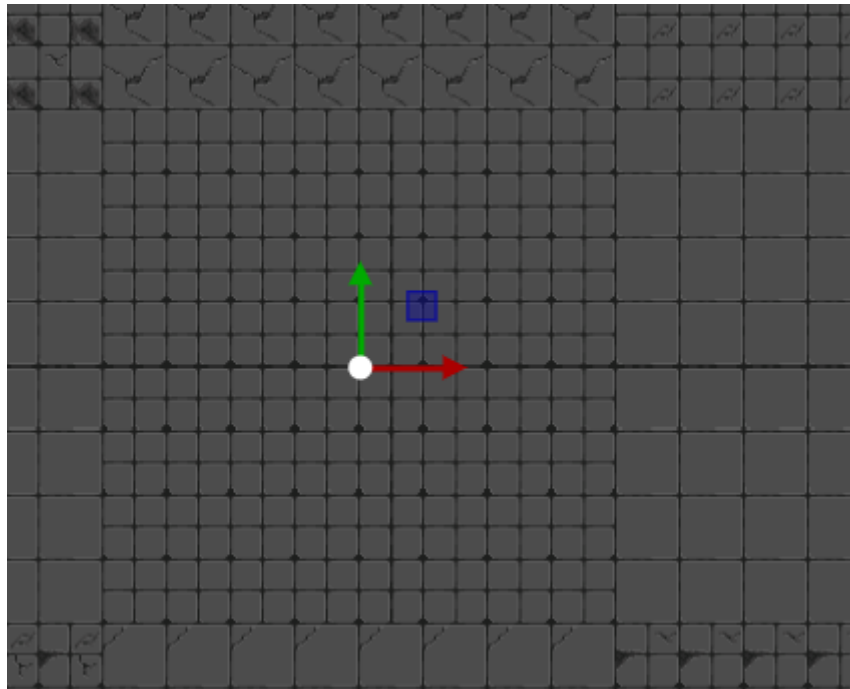
- `void OnCollisionEnter(const Collider& other);`
 - The frame collision happens
 - Physics based collision response still happens
- `void OnCollisionStay(const Collider& other);`
 - When collision continues after the initial contact frame
 - Physics based collision response still happens
- `void OnCollisionExit(const Collider& other);`
 - The frame right after collision has ended
 - Physics based collision response still happens
- `void OnTriggerEnter(const Collider& other);`
 - The frame collision happens
 - No system-based collision response
- `void OnTriggerStay(const Collider& other);`
 - When collision continues after the initial contact frame
 - No system-based collision response
- `void OnTriggerExit(const Collider& other);`

- The frame right after collision has ended
- No system-based collision response

4.9 Lighting

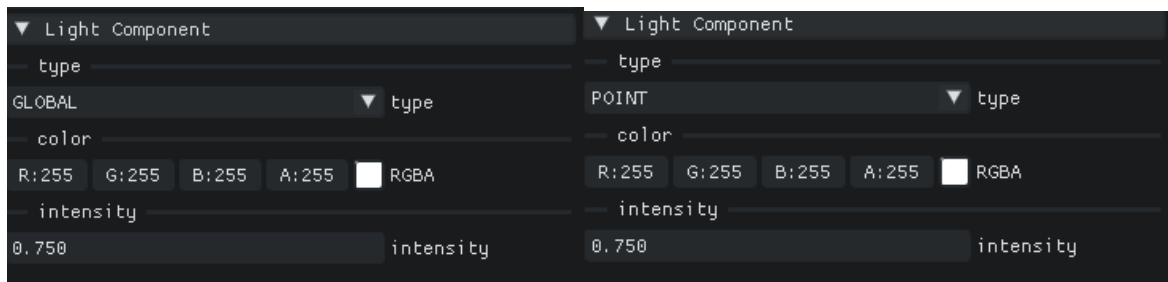
4.9.1 Adding lighting

Lighting settings can be done from the light component which can be added from the inspector window



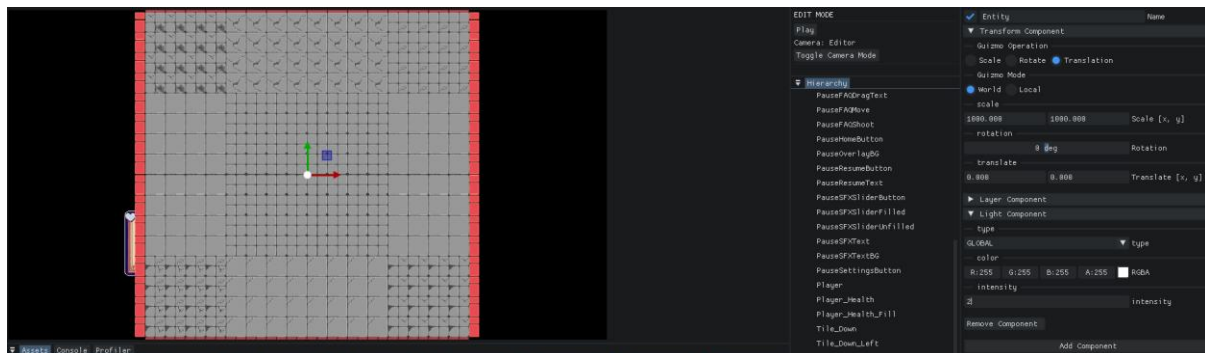
When a light component is added anywhere in a scene without any lighting, the scene will turn completely dark, while generating a light source on the entity of which the light component is attached.

4.9.2 Types of lighting

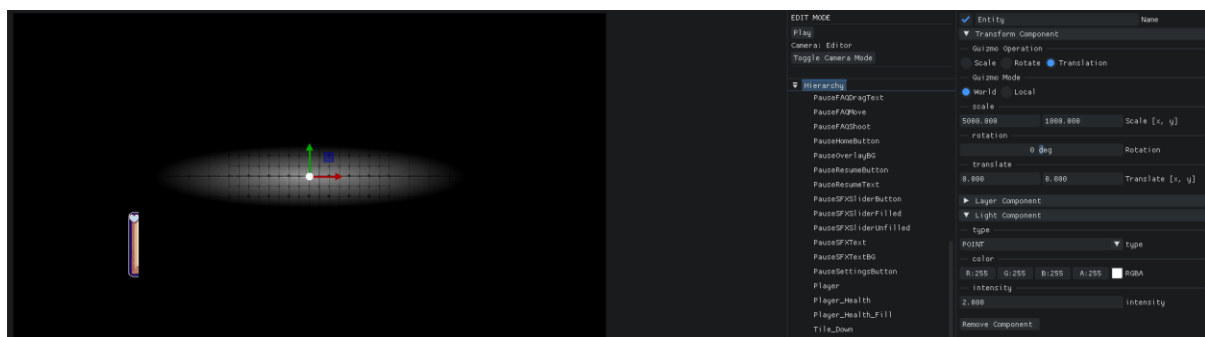


Users have the option between a “global” light and a “point” light.

A global light will apply lighting on the entire scene based on the intensity and color.

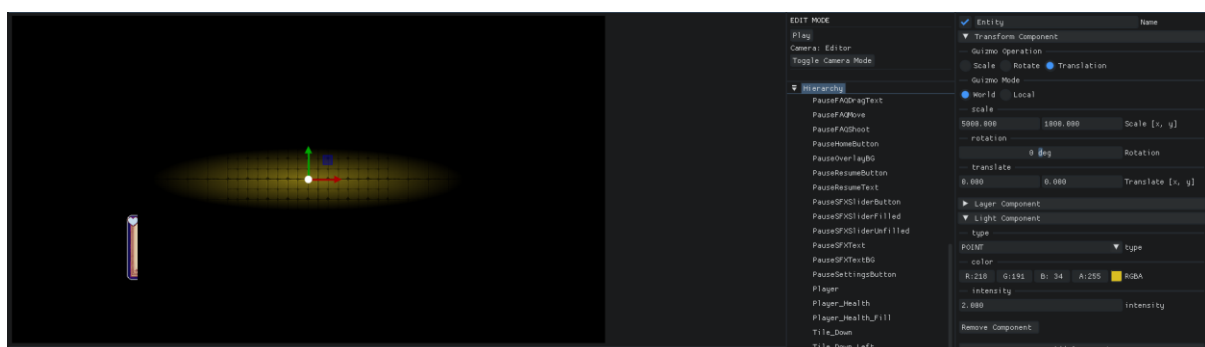


While a point light will apply light bloom, size and position scaling with the transform component of the entity.



4.9.3 Light color

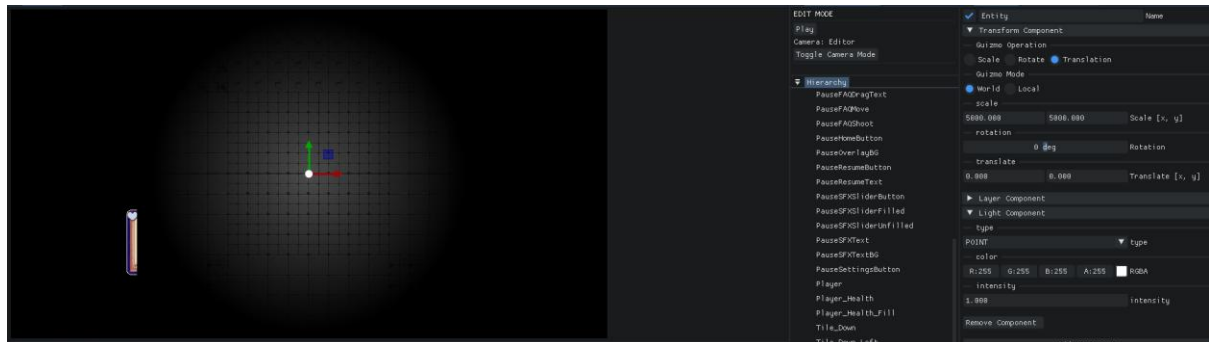
Color affects the color of the lighting effect applied, below we can see a yellow light being applied



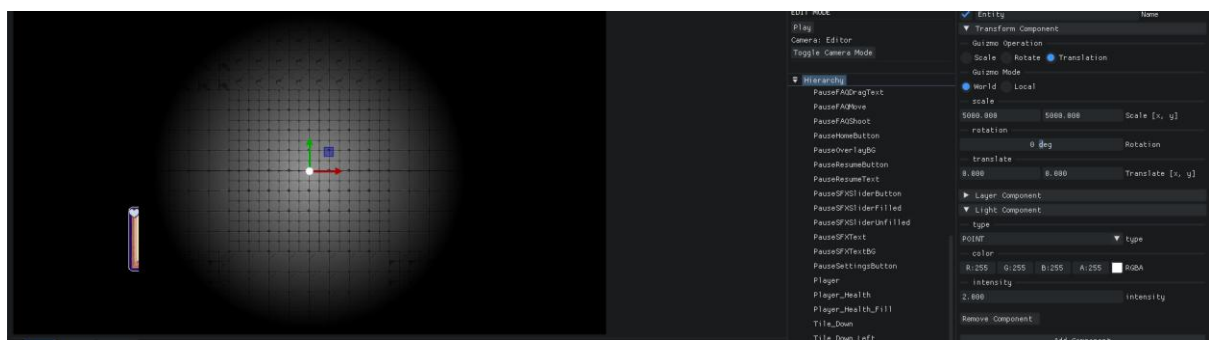
4.9.4 Light intensity

Intensity affects the brightness of the light, allowing for the creation of range of dim to blinding light effects.

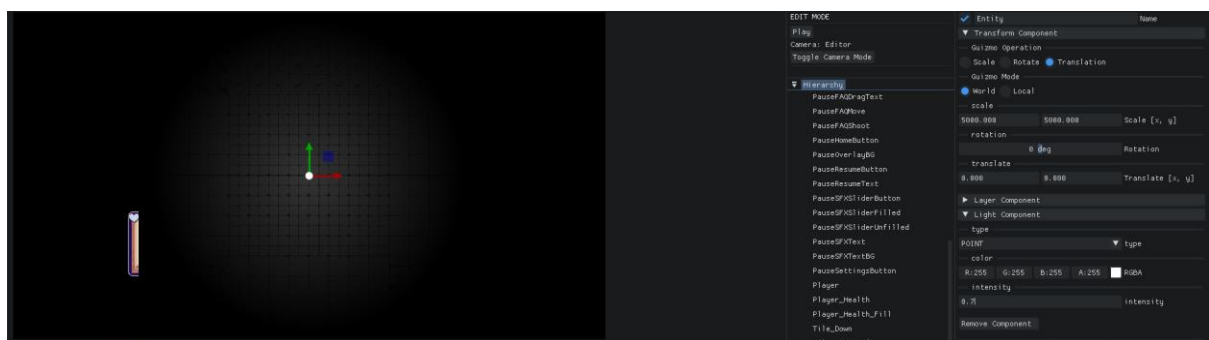
A brightness of 1 is the default brightness of the scene as if it did not have any lighting.



A brightness above 1 is brighter than default



And a brightness below 1 is dimmer than default



4.10 Particle system

The particle system can be added through the inspector by attaching the particleEmitter component to an entity.

The particle system has a large amount of customizability as seen below:



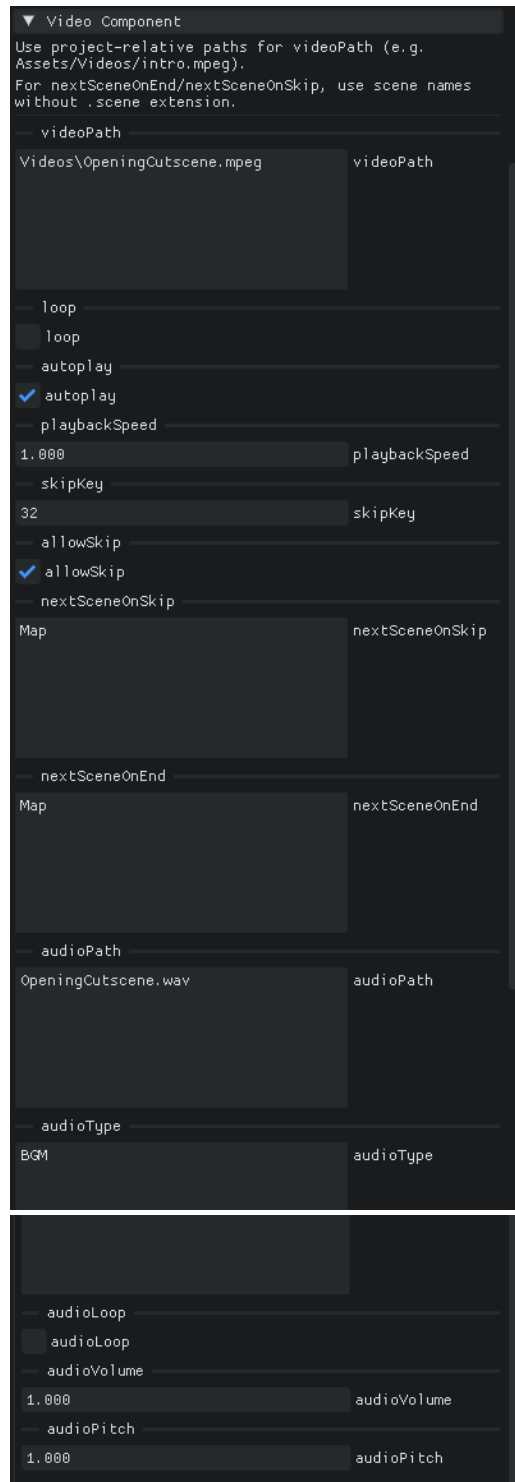
Going through each parameter:

- Rotation range
 - A range of values each particle could randomly rotate towards
- Texture
 - Apply a texture to the particle instead of a solid colored square
- Spread
 - Conical range in degrees (relative to the direction of emission) that the particle could randomly move towards
- Start color
 - Starting color of the particle that the particle will lerp from towards the end color
- Half extents
 - x: width of the possible starting positions of the particles (if emission is a box)
 - y: height of the possible starting positions of the particles (if emission is a box)
- Acceleration
 - Applies acceleration on particles, increasing velocity in specified direction
- End color
 - Ending color of the particle that the particle will lerp towards from the start color
- EmitterPosition
 - Center of the particle emitter
- Start scale
 - Starting size of the particle that the particle will lerp from towards the end size
- End scale
 - Ending size of the particle that the particle will lerp towards from the start size
- Particle lifetime range
 - A range (in seconds) that a given particle will get a random number from to determine how long before it despawns
- Emission rate range
 - A range that changes each frame of how many particles to emit per second
- Particle spawn area shape
 - Circle: spawns particles anywhere in the specified circle radius
 - Box: spawns particles anywhere in the specified bounds (half extents) of the rectangle

- Max particles
 - The maximum amount of particles emitted from this component that can be active at any time
- Radius
 - Radius of the possible starting positions of the particles (if emission is a circle)
- Direction
 - The direction the particles will naturally travel towards
- Min speed
 - Minimum velocity of the particles
- Max speed
 - Maximum velocity of the particles
- Looping
 - Keep emitting particles after specified duration has elapsed
- Duration
 - Duration in seconds to emit particles
- Enabled
 - Enable/disable the emitter
- UseLocalPosition
 - Make position of emission relative to its parent

4.11 Video player

The video player can be added through the inspector by attaching the video component to an entity.



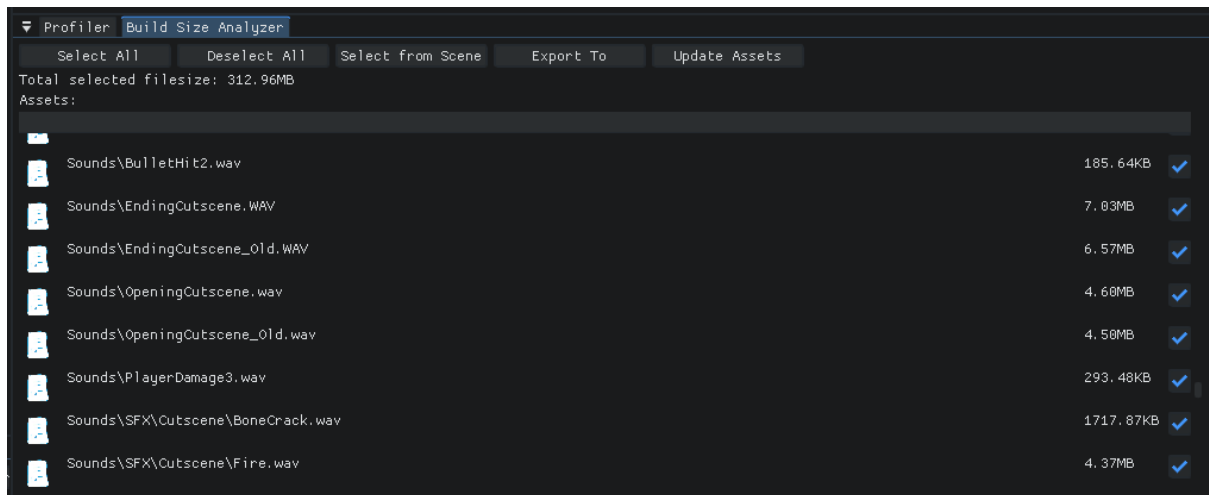
Going through each parameter:

- Video path

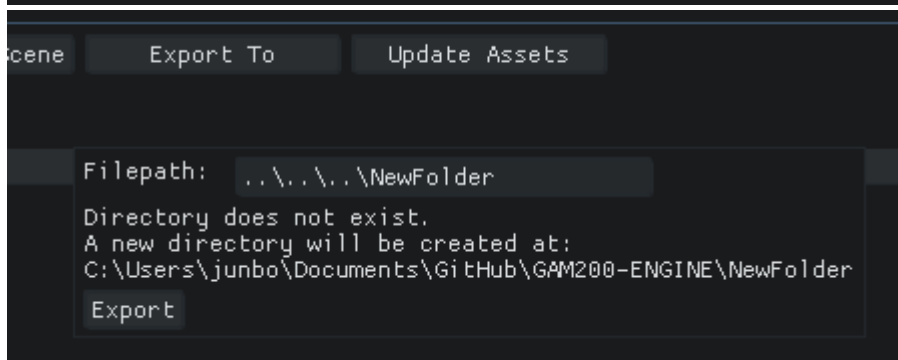
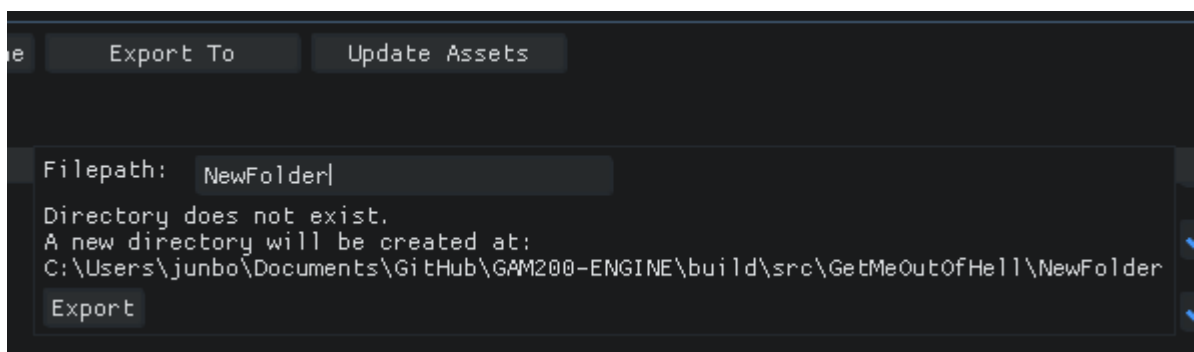
- Path to the video file to be played, format required is specified in the inspector
- Loop
 - Play the video again after it ends
- Autoplay
 - Play the video when the scene is loaded
 - Turning it off will require programatically setting the value to true using scripting to play the video
- Playback speed
 - Playback speed of the video (multiplicative), 1 is the original playback speed of the video
- Skipkey
 - Keycode a user can press to skip the video to the end
- Allowskip
 - Enables/disables skipping of the video
- Next scene of skip
 - Changes the current scene to the specified scene name when the video is skipped
- Next scene of end
 - Changes the current scene to the specified scene name when the video comes to an end
- Audiopath
 - Path to the audio file to be played alongside the video
- Audio type
 - Audio group the audio path is to be set to
- Audio loop
 - Plays the audio again after the end of the video
- Audio volume
 - Volume of the audio file, 1 is the original volume of the audio
- Audio pitch
 - Pitch of the audio file, 1 is the original pitch of the audio

4.12 Build Size Analyzer

The build size analyzer will contain all the assets within the game, alongside with their file sizes. You can select/deselect which assets to export to a folder of choice. Your selection choices are also remembered between launches of the editor.



To choose a folder to export to, you just need to enter in the name of the directory after clicking on the export to button. “..\\” can be used to back out of a directory.



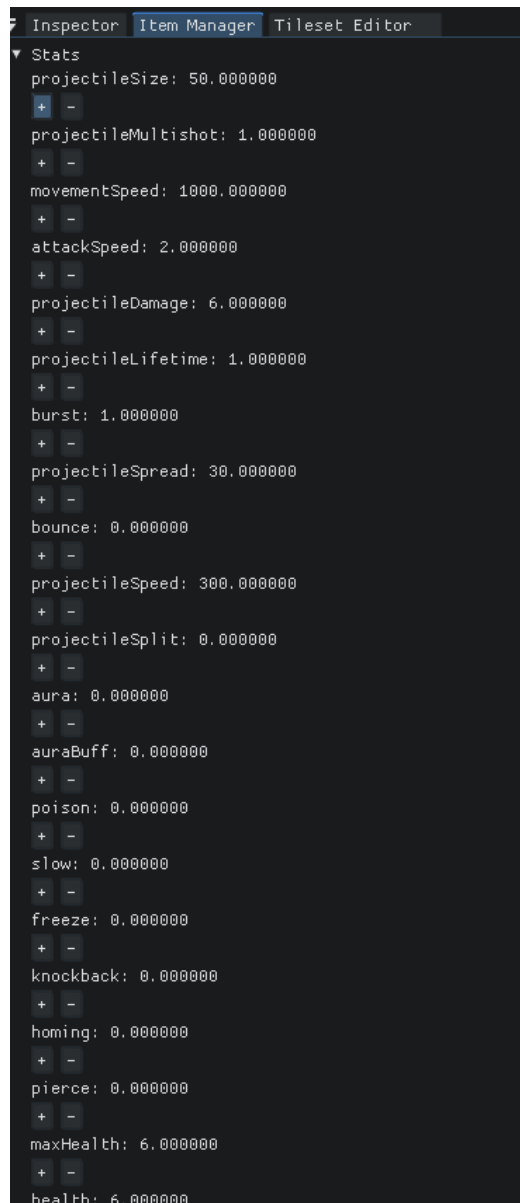
Click the export button to actually export the files. Asset folders that are being used by the editor/the game are not valid export locations due to file access issues.

4.13 Item window

A window that allows the user to add in game items into the player’s active item pool



Also includes a stats menu for the user to directly view the player's stats and modify them



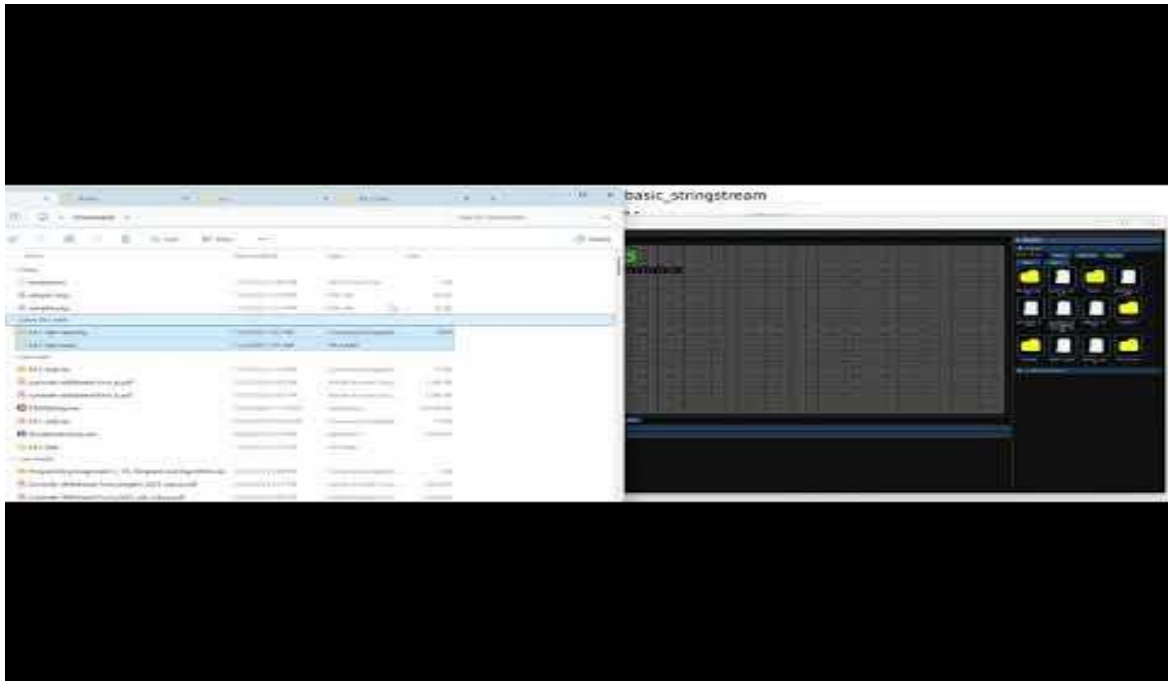
5. Asset Management

Explain how to browse, import, and replace assets using the **Asset Browser** panel.

5.1 Importing

To import assets, MFC needs to be installed in Visual Studio for drag and dropping from file explorer into editor to work. A guide on installing MFC will be provided in section 7. If MFC is installed, the process is simple. Just drag and drop whichever files you wish to put into assets browser.

Textures will need to be placed in the Textures folder before our ResourceManager internally will be able to load the textures.



[Link to YouTube video](#)

5.2 Browsing

Clicking on the folder icon brings you into the sub-directory. You can also drag and drop any asset into a folder to move them into the sub-directory

In a sub-directory, there will be a flipped folder icon with the name “\..”. Clicking on that folder will bring out to the parent directory, but more importantly it serves as a drag and drop target for moving files to the parent directory.

[Link to YouTube Video](#)

5.3 Replacing Assets

Using the object picking functionality, if an object is selected and it has the SpriteRendererComponent, simply dragging and dropping a texture from the asset browser into the viewport window will replace the current texture of the object into the new one.

[Link to YouTube Video](#)

6. Troubleshooting / FAQ

Q: My editor camera is not moving.

A: Ensure the editor window is focused and you are using right-click + mouse drag.

Q: Objects disappear when I switch over to Instance rendering.

A: Play and then pause the desired scene to allow the engine to update the instance buffer and have the objects rendered with instancing.

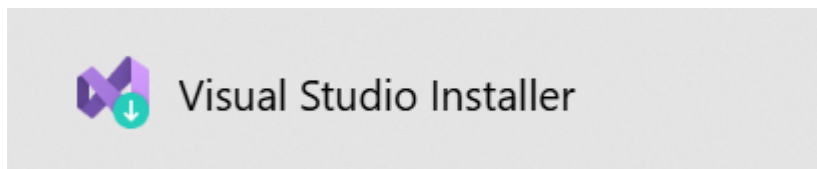
Q: ImGuizmos does not show up when I select an object.

A: Ensure the Inspector window is open and that the selected object has a TransformComponent.

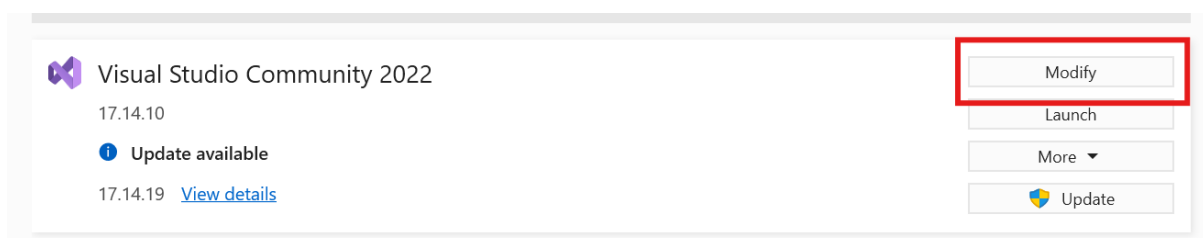
Q: How do I install MFC?

A:

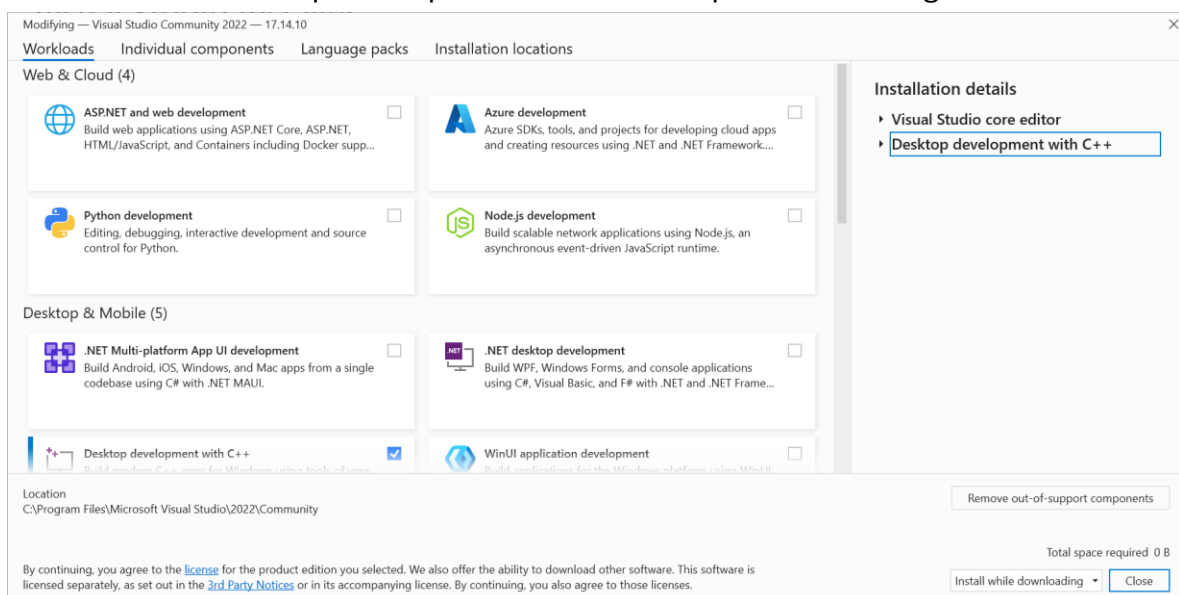
1: Open Visual Studio Installer



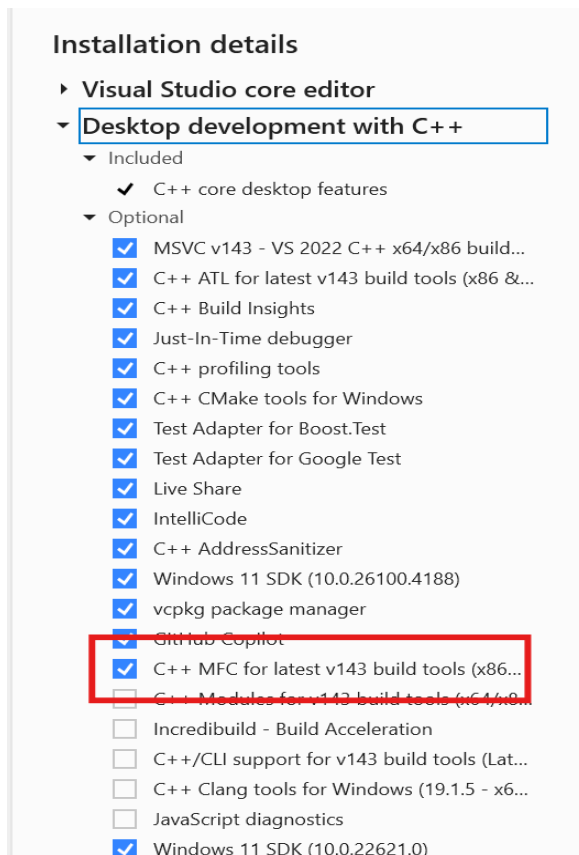
2: Click the modify button on the right of the version you are currently using



3: Find for the “Desktop development with C++” dropdown on the right



4: Click on the C++ MFC for latest... option



5: Select the modify key on the bottom right of the popup. (Visual studio will need to be closed while the installation is happening).

6: Recompile the project after it is finished.